Machine Learning for Finance

Supervised Learning - NonLinear Regression

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
   - Regression Trees
   - Bagging
   - Random Forests
5. Gradient Boosting
6. Model Comparison

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
5. Gradient Boosting
6. Model Comparison

Case Study: Predicting Cross-Sectional Returns

Suppose we want to predict asset returns using firm characteristics.
Target variable:

$$r_i$$

Feature vector:

$$X_i = (X_{1i}, X_{2i}, \ldots, X_{Ki})$$

Goal:

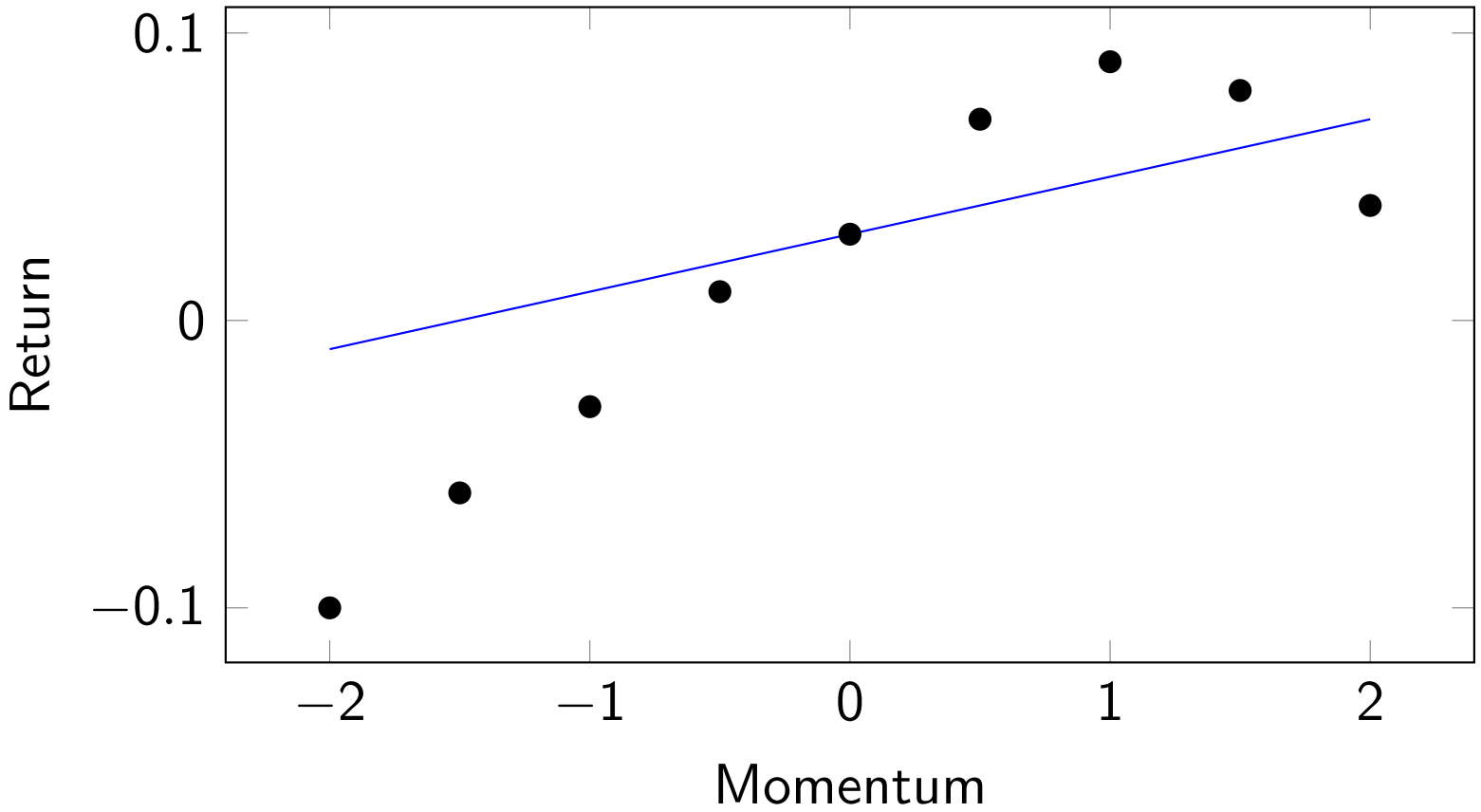
$$r_i = f(X_i)$$

A common assumption is that the relationship is linear:

$$r_i = a + bX_{1i} + cX_{2i}$$

Exploratory Data Analysis

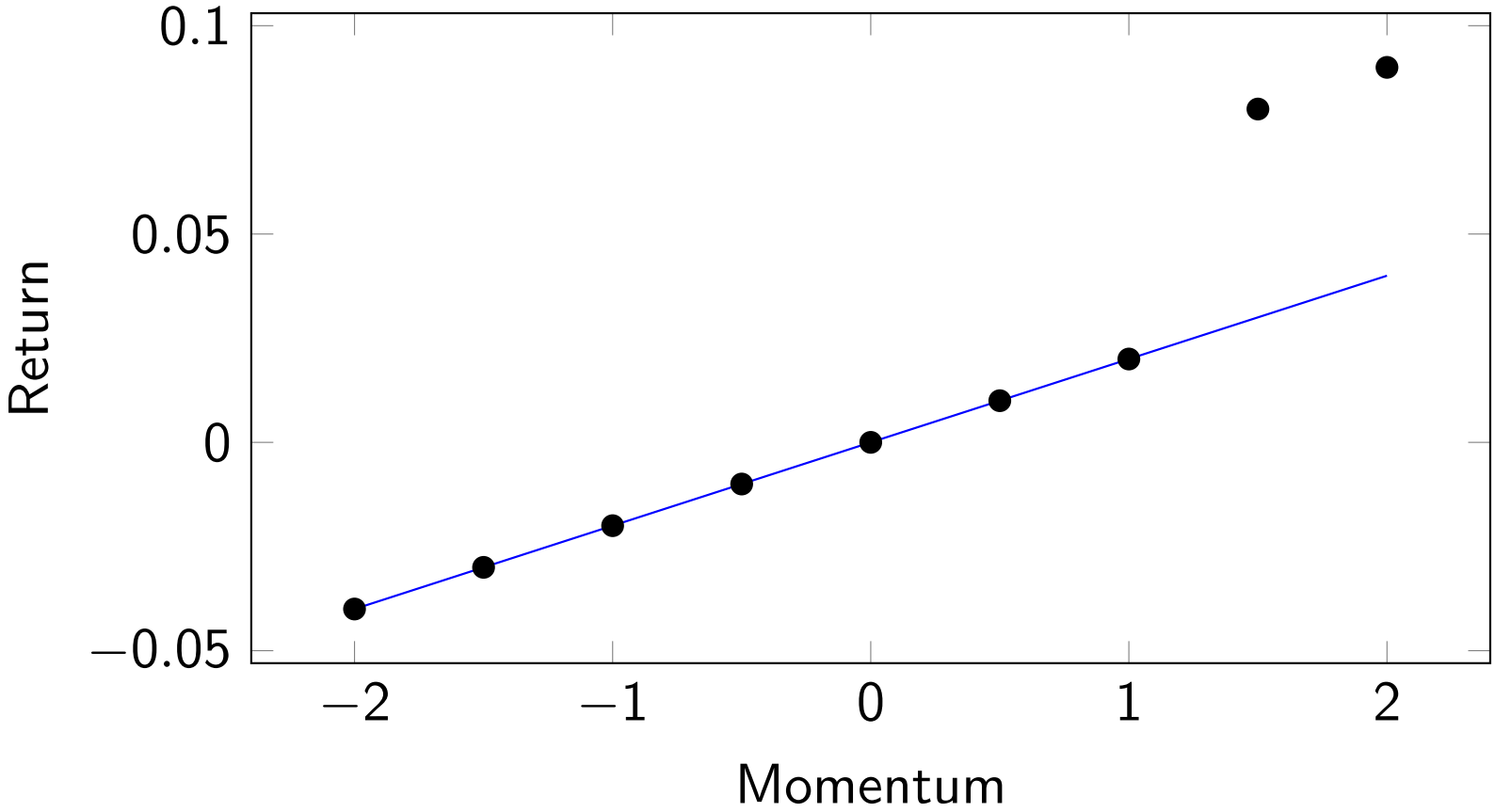
Suppose exploratory data analysis reveals the following pattern.



The linear model may not capture the nonlinear relationship.

Another Possible Nonlinearity

In some cases the relationship may exhibit threshold effects.



The relationship may change abruptly once a threshold is crossed.
Linear models may struggle to capture such jumps.

Frame Title

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
5. Gradient Boosting
6. Model Comparison

OLS: Linear in Parameters, Not Necessarily in X

A familiar OLS regression:

$$r_i = \alpha + \beta_1 X_i + u_i$$

We can also estimate nonlinear transformations using OLS:

$$r_i = \alpha + \beta_1 X_i + \beta_2 X_i^2 + u_i$$

Q: Is the second regression still linear?
A: Yes, it is *linear in the unknown parameters* α, β_1, β_2, even though its fitted relationship can curve in X.

Linearity in parameters $\neq$ linearity in the original variable.

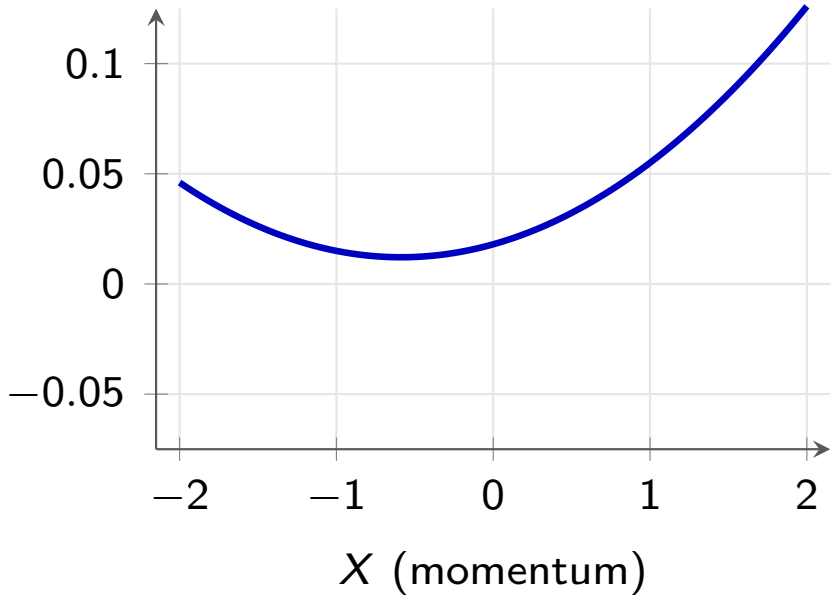
Nonlinear OLS: We Choose the Transformation

Suppose we suspect a nonlinear relationship between momentum and returns.

We might fit a quadratic:

$$\hat{r}_i = \hat{\alpha} + \hat{\beta}_1 X_i + \hat{\beta}_2 X_i^2$$

Or we could try a cubic, log, threshold, or another transformation.



Q: What limitation remains?
A: We must specify the functional form *before* estimating the coefficients. What if the true shape is not quadratic?

From Nonlinear OLS to a GAM

Instead of deciding in advance that the effect of momentum is quadratic, let its shape be a flexible function:

$$r_i = \alpha + f(X_i) + u_i.$$

The function is approximated using **basis functions**:

$$f(X_i) = \theta_1 B_1(X_i) + \theta_2 B_2(X_i) + \cdots + \theta_m B_m(X_i).$$

Q: What is new, and what stays the same?
A: We select a flexible *family* of transformations rather than specifying one polynomial shape. We still estimate coefficients linearly.

The basis type, its size, and the degree of smoothing are modeling choices; a GAM does not discover an unrestricted function automatically.

Polynomial OLS Is a Basis Expansion

Return to our familiar quadratic OLS model:

$$r_i = \alpha + \beta_1 X_i + \beta_2 X_i^2 + u_i.$$

We can write it in basis-function notation by defining

$$B_1(X) = X, \qquad B_2(X) = X^2,$$

so that

$$r_i = \alpha + \beta_1 B_1(X_i) + \beta_2 B_2(X_i) + u_i.$$

Q: What changes when we replace these two functions with a richer spline basis?

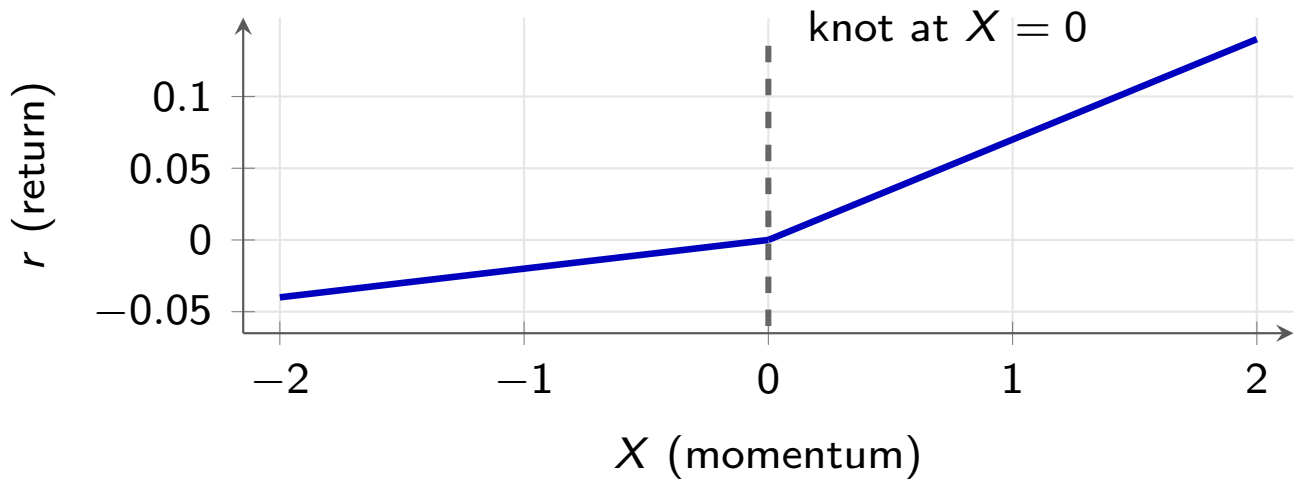
A: The fitted relationship can take many more shapes, without requiring us to choose a specific quadratic (or cubic) polynomial in advance.

In the broad additive representation, polynomial OLS is a special case. In practice, GAMs commonly use richer, penalized smooths.

Piecewise OLS: A Single Knot

A simple nonlinear OLS model allows the momentum slope to change at $X = 0$:

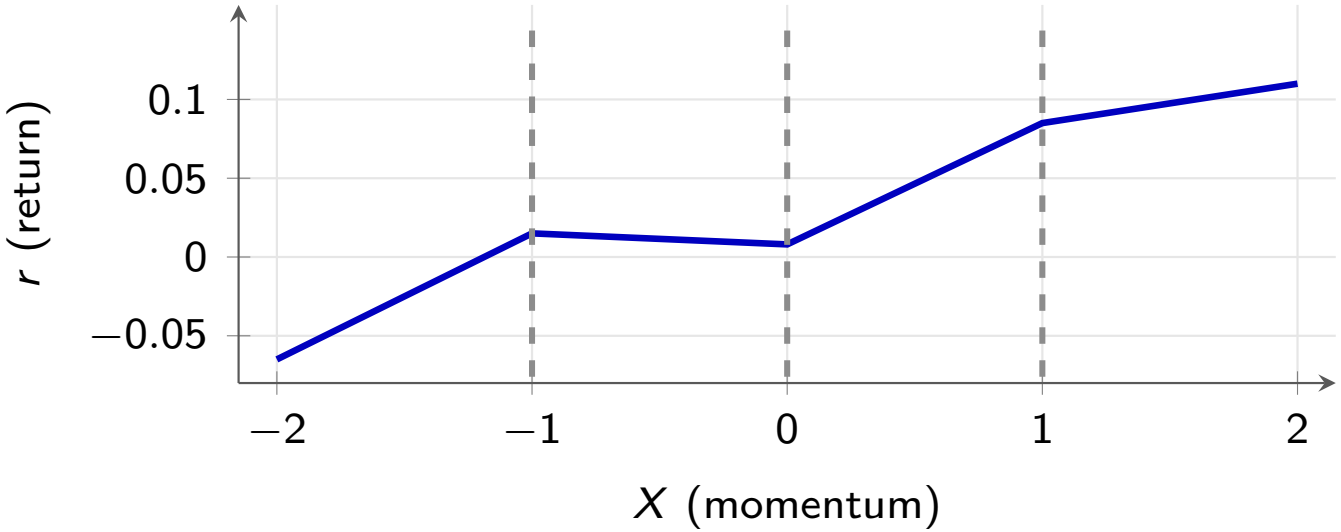
$$r_i = \alpha + \beta_1 X_i + \beta_2 (X_i)_+ + u_i, \qquad (X_i)_+ = \max\{X_i, 0\}.$$



Q: What are the slopes to the left and right of the knot?
A: Left: β_1. Right: $\beta_1 + \beta_2$. The fitted curve changes slope, but the regression remains linear in its parameters.

Adding Knots Increases Flexibility

With several knots, the fitted relationship can change slope in several regions.



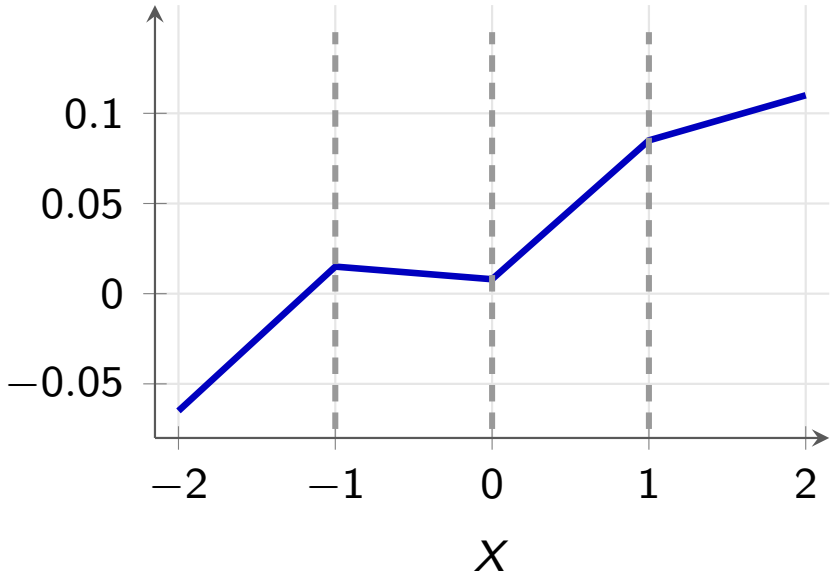
Q: What assumption does this model make at each knot?

A: The marginal effect of momentum changes abruptly at each knot. This may be appropriate for genuine thresholds, but otherwise we may prefer slopes that change gradually.

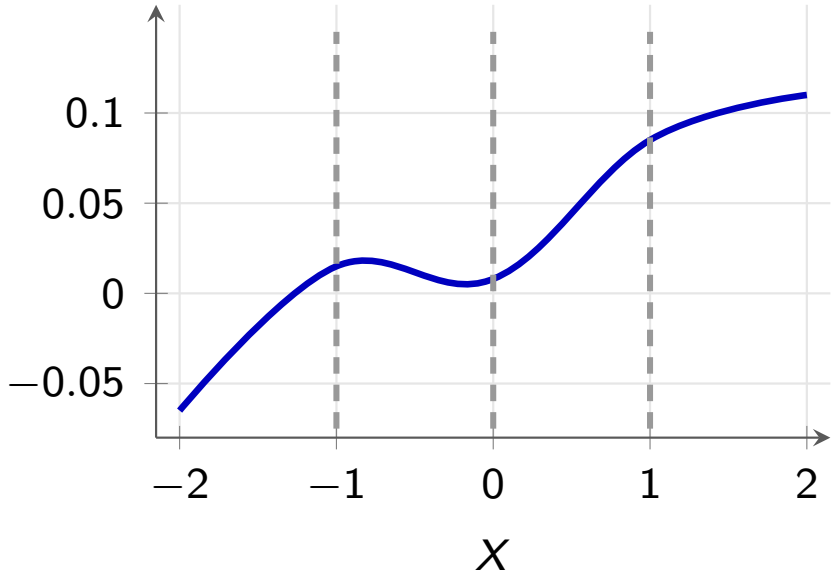
Spline Intuition: Join Pieces Smoothly

A spline joins simple polynomial pieces at selected locations (*knots*). Cubic splines can join the pieces without abrupt changes in slope.

Piecewise linear: corners



Smooth joins



Q: Does a knot need to create a visible corner?

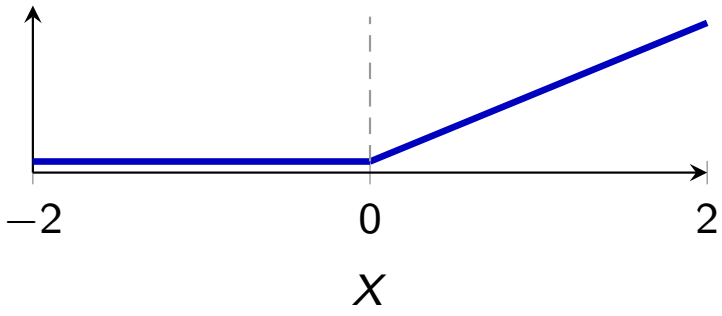
A: No. Cubic spline pieces are constructed to agree in fitted value, slope, and second derivative at interior knots.

Schematic curves illustrating smoothness, not estimates from data.

How Cubic Splines Achieve Smooth Joins

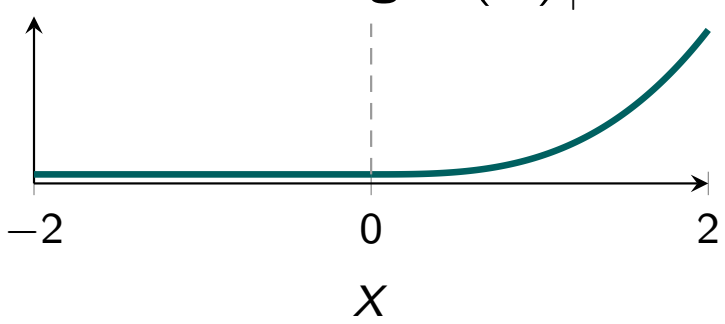
Recall the hinge used in piecewise OLS: $(X)_+ = \max\{X, 0\}$.

Linear hinge: $(X)_+$



The slope jumps at $X = 0$.

Cubic hinge: $(X)_+^3$



The slope changes smoothly at $X = 0$.

Add the cubic hinge to a cubic OLS model:

$$f(X) = \alpha + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \gamma(X)_+^3.$$

Q: Why is the join smooth at $X = 0$?
A: The cubic hinge, its slope, and its second derivative are all zero at the knot. The two polynomial pieces therefore join smoothly.

Basis Functions: Transformed Predictors

A spline can be represented by a collection of predetermined functions $B_1(X), \ldots, B_m(X)$:
For example, X, X^2, X^3, and $(X - \kappa)^3_+$ at knots κ can form a cubic spline basis.

$$f(X_i) = \theta_1 B_1(X_i) + \cdots + \theta_m B_m(X_i).$$

Define new predictors $Z_{ji} = B_j(X_i)$. Then the GAM can be written

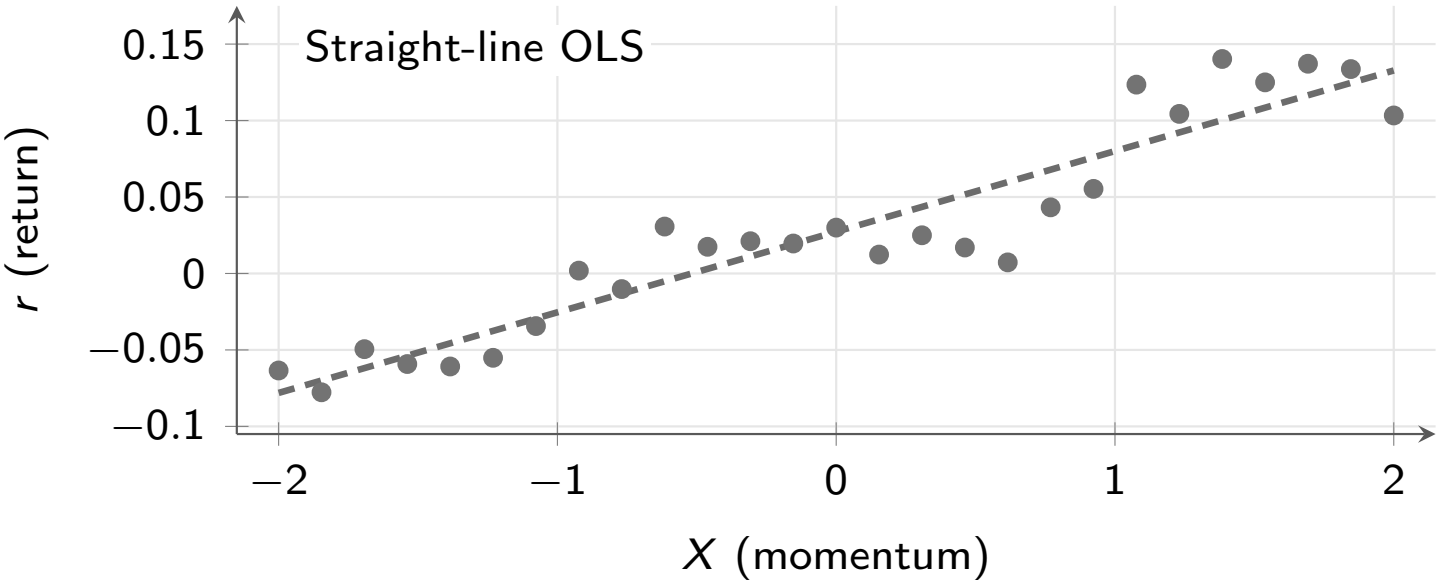
$$r_i = \alpha + \theta_1 Z_{1i} + \cdots + \theta_m Z_{mi} + u_i.$$

Q: What familiar estimation problem does this resemble?
A: OLS with transformed features. The $B_j(\cdot)$ specify *columns of the design matrix*; the θ_j are coefficients we estimate.

The transformation is nonlinear in X; the model remains linear in θ.

The Relationship We Want to Estimate

Suppose we observe momentum and returns. The hypothetical data suggest the relationship may be nonlinear.

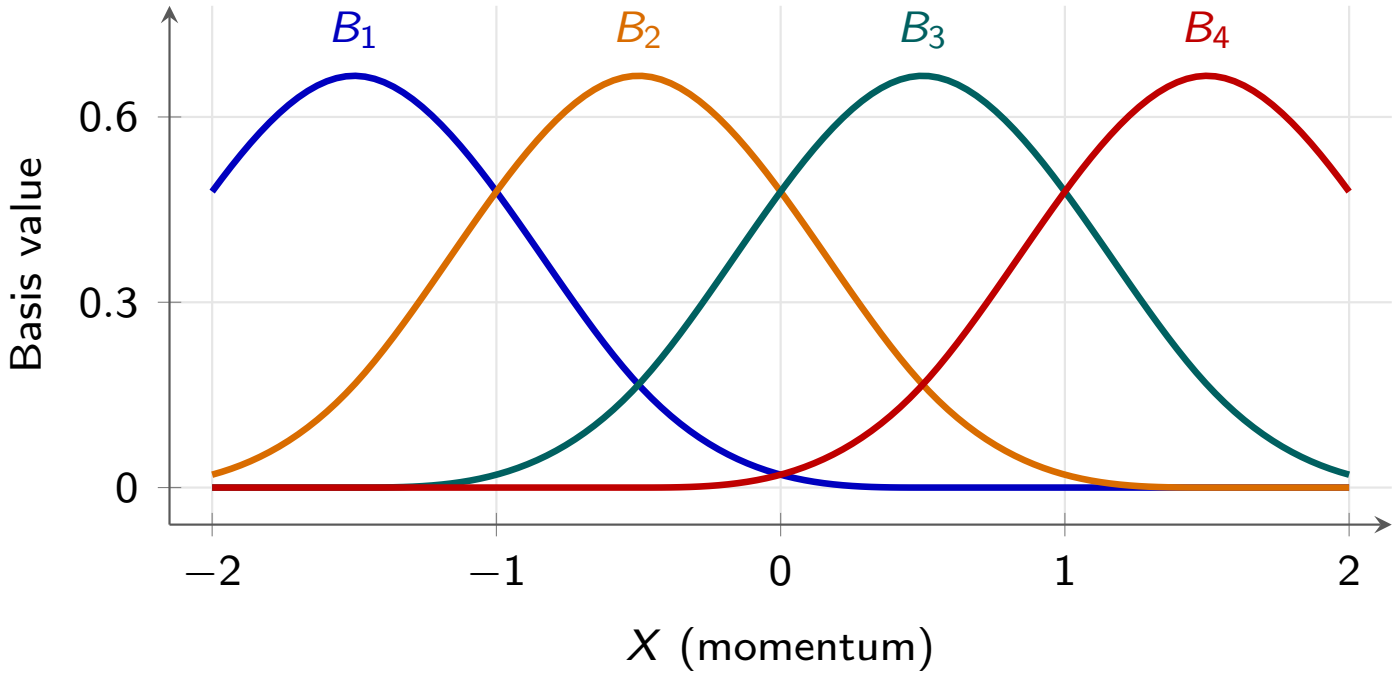


Q: How can we capture the curvature without imposing a quadratic or cubic form?

A: Use flexible basis functions, and let regression estimate how much each contributes.

Spline Basis Functions: Building Blocks

We choose four illustrative *smooth spline basis functions* of momentum.



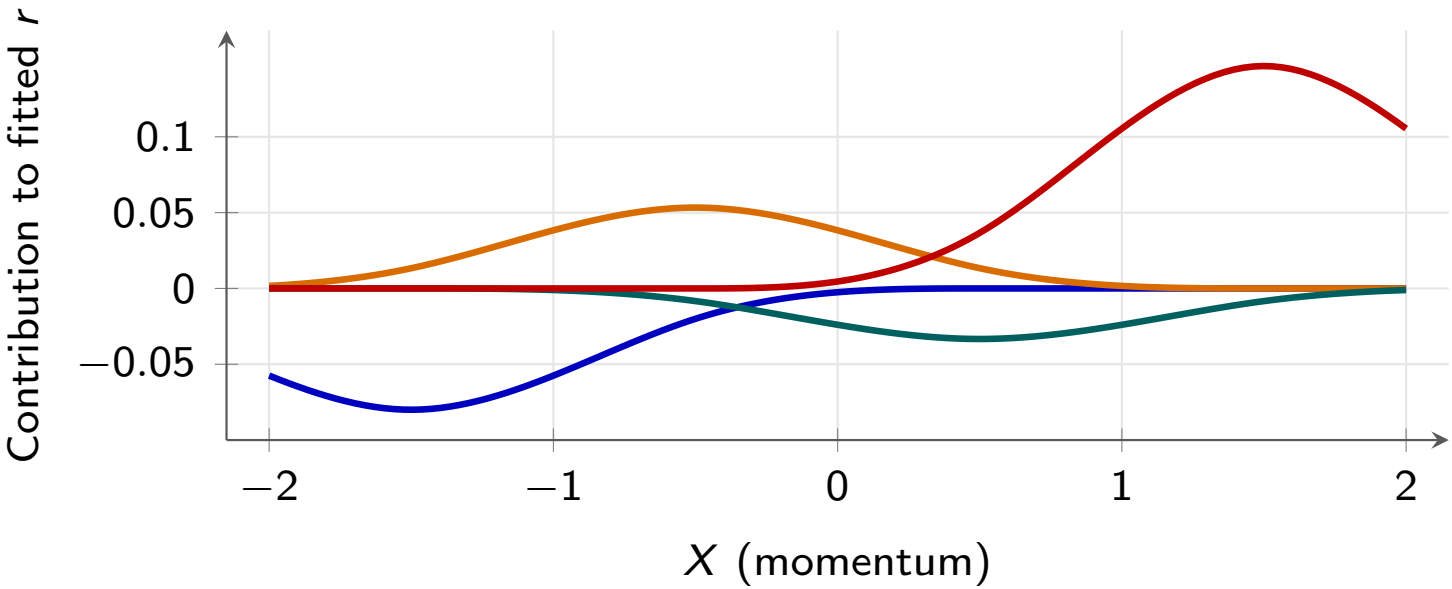
Q: Are these four curves separate predictions of returns?

A: No. They are predetermined transformed predictors. We estimate their coefficients from the observed data.

Weighting the Basis Functions

Least squares on our illustrative observations gives these coefficients:

$$\hat{f}(X) = -0.12B_1(X) + 0.08B_2(X) - 0.05B_3(X) + 0.22B_4(X).$$



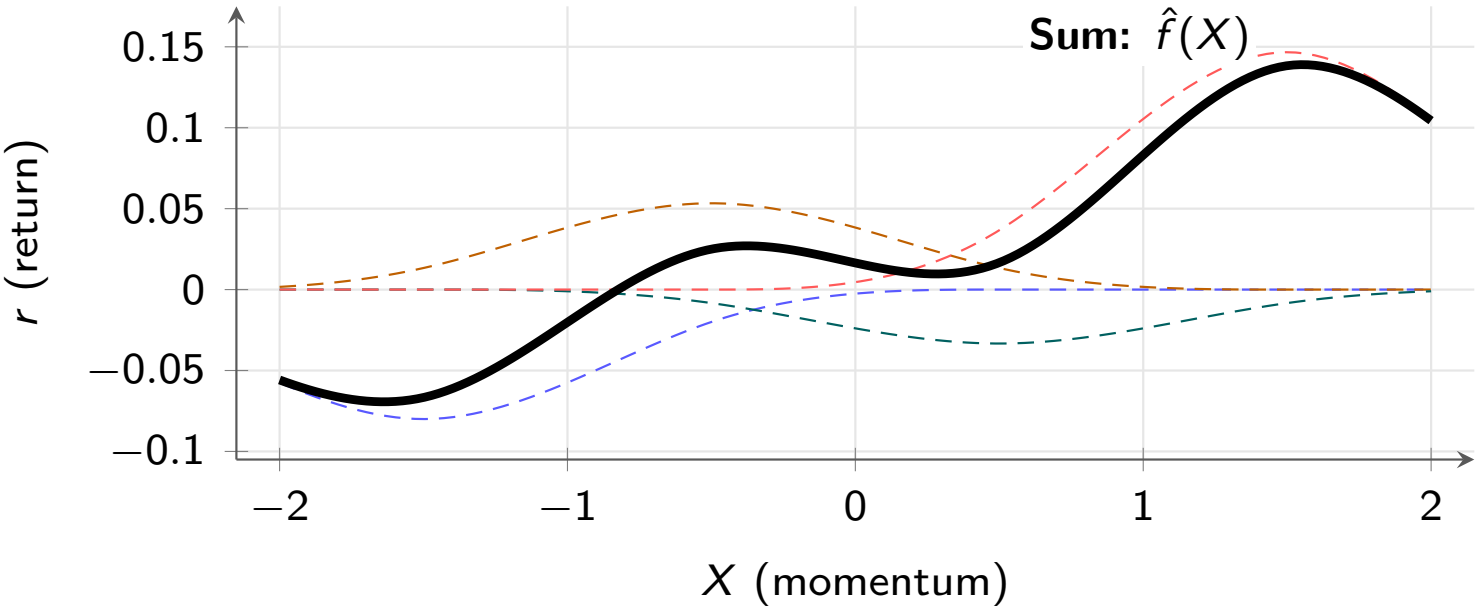
Q: What does a negative coefficient do?

A: It makes that basis function contribute negatively to fitted returns.

The Sum of the Components Is the Fitted Curve

The four weighted components are *added*, not used as separate predictions.

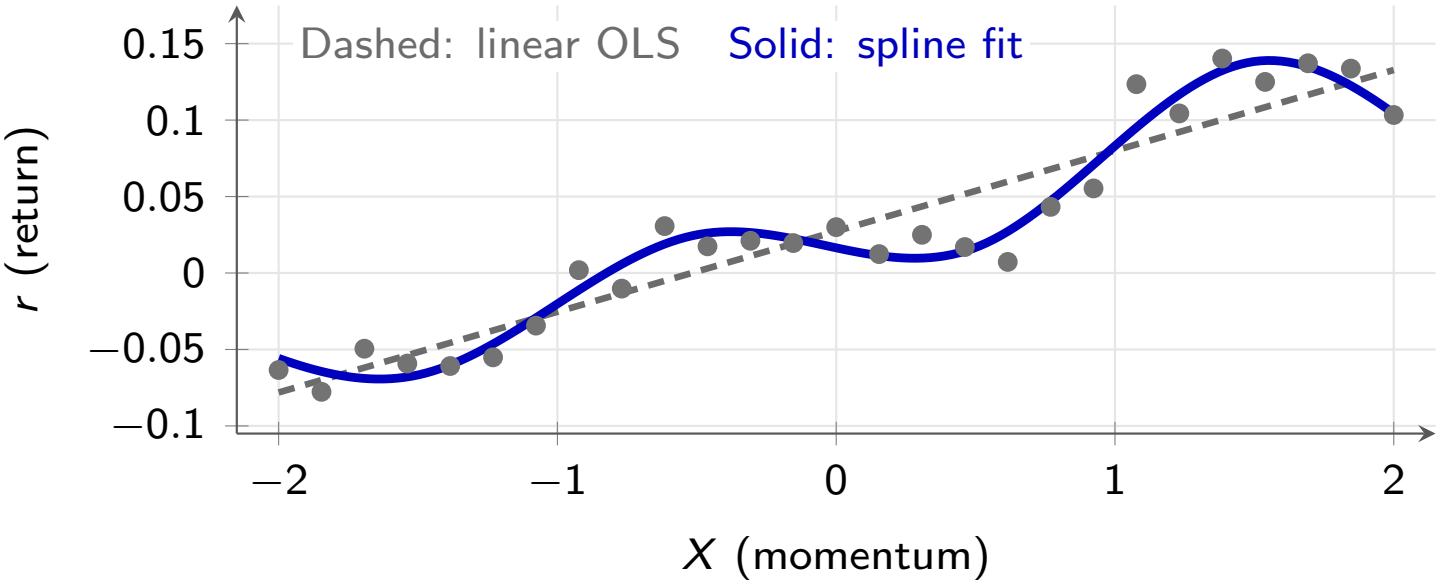
$$\hat{f}(X) = \sum_{j=1}^{4} \hat{\theta}_j B_j(X)$$



A smooth nonlinear fitted relationship emerges from a linear combination.

The Fitted Curve and the Observed Data

Return to *exactly the same observations* from the opening slide.



Q: What have we accomplished?

A: We captured a nonlinear association without selecting a single polynomial shape in advance, *while remaining linear in the coefficients*.

Estimating the Coefficients: Still Linear Regression

For each observed momentum value X_i, the basis values $B_1(X_i), \ldots, B_m(X_i)$ are just known regressors.

$$r_i = \alpha + \sum_{j=1}^{m} \theta_j B_j(X_i) + u_i.$$

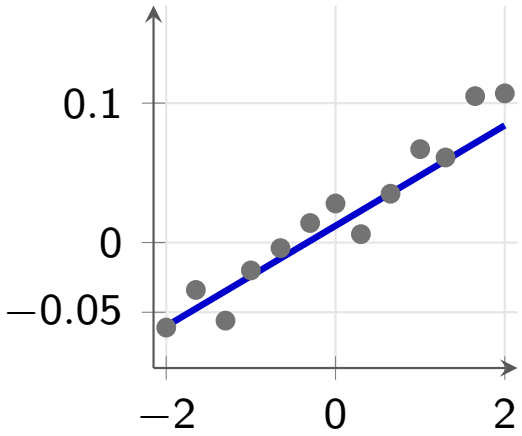
Estimate the coefficients by least squares:

$$\min_{\alpha, \boldsymbol{\theta}} \sum_{i=1}^{n} \left[r_i - \alpha - \sum_{j=1}^{m} \theta_j B_j(X_i) \right]^2.$$

More Flexibility Can Also Mean Overfitting

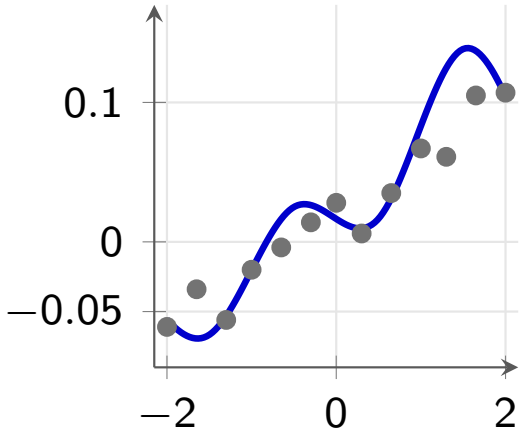
Suppose we observe the same noisy momentum–return data and fit three curves.

Too rigid



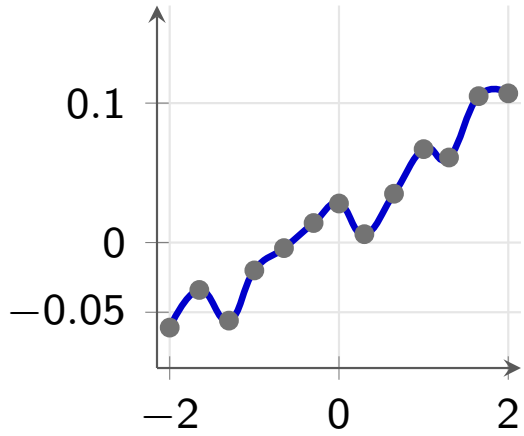
Underfits curvature

Smooth fit



Captures main shape

Too wiggly



Follows sampling noise

Q: Why not just use enough basis functions to fit every observation?

A: Excellent in-sample fit can produce poor predictions on new data.

Penalized Least Squares Controls Smoothness

Smooth joins at knots are built into the cubic spline basis. A commonly used GAM objective also penalizes excessive curvature between knots:

$$\min_{\alpha,\boldsymbol{\theta}} \underbrace{\sum_i (r_i - \hat{r}_i)^2}_{\text{fit the data}} + \lambda \underbrace{\int [f''(x)]^2 \, dx}_{\text{penalize wiggliness}} .$$

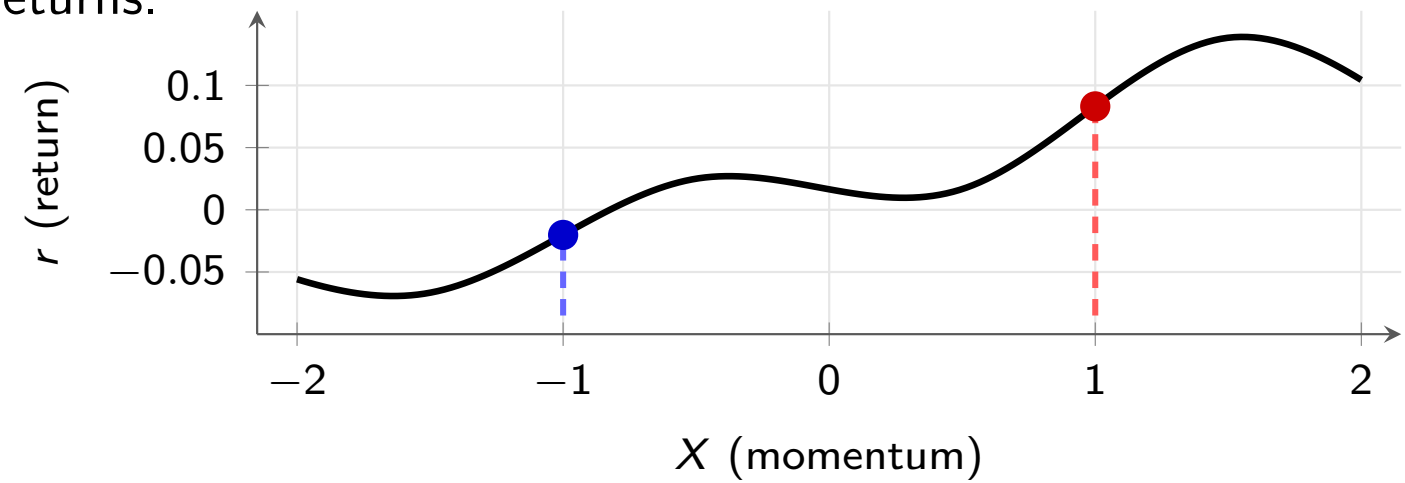
Q: What happens as λ increases?
A: Curvature becomes more costly, so the fitted function usually becomes smoother (potentially too smooth).

- Small λ: flexible fit; greater risk of overfitting.
- Large λ: smoother fit; greater risk of underfitting.
- In practice, select smoothing strength using a data-driven criterion (e.g., REML or cross-validation).

This is still a penalized regression problem in the basis coefficients. With a quadratic curvature penalty, the objective is quadratic in $\boldsymbol{\theta}$.

Interpreting the Estimated Momentum Effect

Once estimated, the function $\hat{f}(X)$ gives the fitted contribution of momentum to expected returns.



Q: How should we interpret $\hat{f}(1) - \hat{f}(-1)$?

A: It is the fitted change in the contribution of momentum as X moves from -1 to 1, holding other additive terms fixed.

This is an association in the model, not automatically a causal effect. The curve is hypothetical.

Why Is It Called an Additive Model?

With momentum, volatility, and another predictor, we might estimate

$$r_i = \alpha + f_1(\text{Momentum}_i) + f_2(\text{Volatility}_i) + f_3(X_{3i}) + u_i.$$

Each predictor has its own flexible effect, but those effects are **added** to make the fitted value.

Q: Does the shape of the momentum effect automatically change when volatility changes?

A: No. The basic additive specification has no interactions. We must add an interaction smooth if the shape of one effect should depend on another variable.

We are using a Gaussian GAM with an identity link here. More general GAMs can use other response distributions and link functions.

GAMs: Main Takeaways

1. **OLS already handles nonlinear predictors:** polynomials, interactions, and piecewise transformations are allowed.
2. **A GAM builds more flexible transformations:** its smooth functions are represented by sums of basis functions.
3. **The coefficients still enter linearly:** the basis-function values are transformed regressors, just as in OLS.
4. **Smoothness is controlled:** penalties discourage overfitting by overly wiggly functions.

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
5. Gradient Boosting
6. Model Comparison

Motivation: Predicting Returns with Momentum

Q: Can the return–momentum relationship be nonlinear?

Consider predicting next-period returns across stocks using information available at t:

$$\widehat{r}_{i,t+1} = f(\mathrm{Mom}_{i,t}, \mathrm{Vol}_{i,t}; \theta)\,.$$

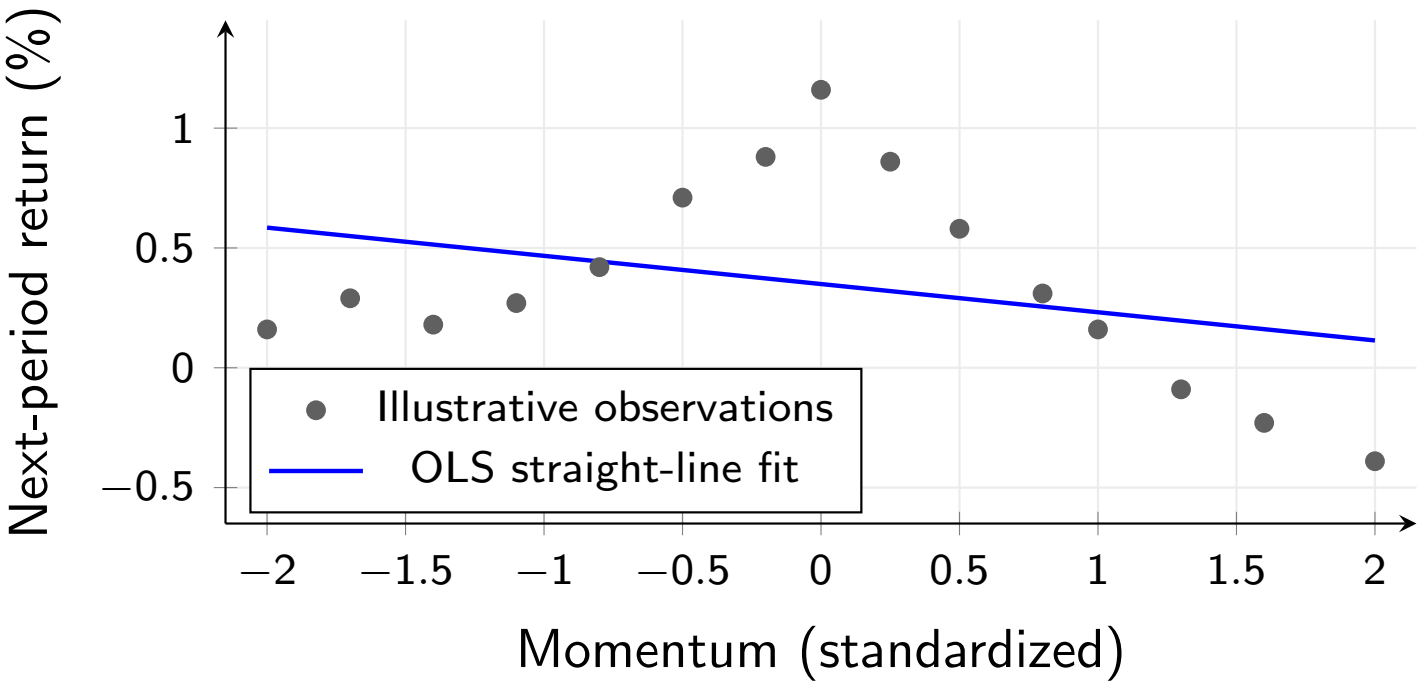
- $\mathrm{Mom}_{i,t}$: a momentum characteristic for stock i.
- $\mathrm{Vol}_{i,t}$: historical volatility for that stock.
- θ: parameters learned from historical stock observations.

Our question is **cross-sectional**: How do characteristics observed at t relate to differences in returns at $t+1$? We are not fitting an autoregressive model of a return series.

A Nonlinear Momentum Relationship

Q: Why might a straight-line fit be restrictive?

Momentum may predict higher returns over some range, but the relationship could weaken or reverse for extreme momentum values.



Illustrative data, not an empirical finding about momentum.

Nonlinearity Is Not Unique to Neural Networks

Q: Can't OLS and GAMs already fit nonlinear relationships?

Yes. OLS can use nonlinear transformations of the features:

$$E[r \mid M] = \alpha + \beta_1 M + \beta_2 M^2.$$

A conventional additive GAM can learn flexible nonlinear component functions:

$$E[r \mid M, V] = \alpha + f_M(M) + f_V(V).$$

For spline-based GAMs, we choose a family of basis functions and estimate their coefficients:

$$f_M(M) = \sum_{j=1}^{J} \gamma_j B_j(M).$$

Neural networks go further: Rather than specifying nonlinear transformations and interactions in advance, we let the network learn useful transformations and combinations of features from the data.

From GAMs to Neural Networks

Q: What changes when the basis functions are learned?

For a GAM, the basis functions $B_j(M)$ are typically specified before estimating their coefficients. A hidden unit in a neural network instead computes

$$h_j(X) = \sigma\left(w_j^\top X + b_j\right), \qquad X = (M, V)^\top.$$

The network estimates **both** the weights and biases (w_j, b_j) defining the transformations, and the output weights v_j:

$$\widehat{r} = c + \sum_{j=1}^{H} v_j h_j(X).$$

- GAM: usually a predetermined basis, with estimated coefficients.
- Network: the shapes and combinations of hidden features are learned.

Standard additive GAMs omit interactions unless explicitly added; GAMs can also be extended to include interactions.

Linear Regression as a Special Case

Q: How is OLS related to a neural network?

A linear prediction rule is

$$\widehat{r}_i = \alpha + \beta^\top X_i.$$

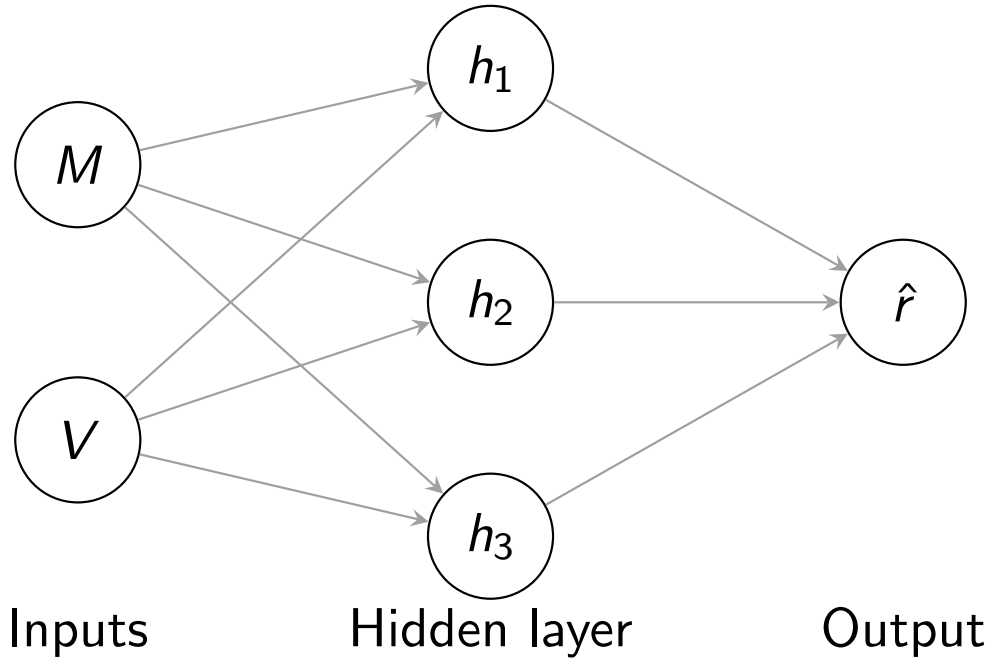
We can write the same rule in network notation:

$$\widehat{r}_i = b + w^\top X_i, \qquad b = \alpha, \quad w = \beta.$$

- This is a network with **no hidden layer** and a linear output.
- Adding linear layers alone does **not** create nonlinearity: a composition of affine mappings remains affine.
- Nonlinear **activation functions** are essential to capturing nonlinearities in the data.

A Feedforward Neural Network

Q: How do the two characteristics become a predicted return?



- Each arrow has a learned **weight**; each neuron also has a **bias**.
- Hidden units create intermediate features of momentum and volatility.
- The output combines them into a **continuous return prediction**.

What Does a Hidden Unit Compute?

For stock i, with $X_i = (M_i, V_i)^\top$:

$$\underbrace{a_{ij}}_{\text{input score}} = w_{jM} M_i + w_{jV} V_i + b_j, \qquad \underbrace{h_{ij}}_{\text{hidden output}} = \sigma(a_{ij}).$$

- w_{jM}, w_{jV}: weights on the two characteristics for neuron j.
- b_j: bias; it shifts the neuron's response.
- $\sigma(\cdot)$: activation function, typically nonlinear.

For a single hidden layer:

$$\widehat{r}_i = c + \sum_{j=1}^{H} v_j h_{ij}.$$

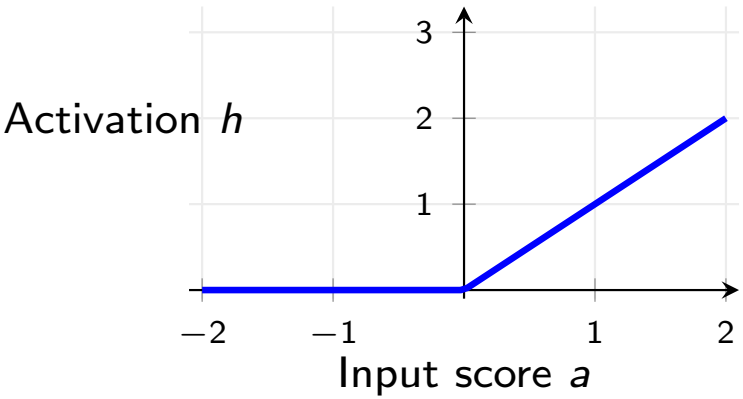
The parameters $\{w_{jM}, w_{jV}, b_j, v_j, c\}$ are learned jointly.

The ReLU Activation Function

Q: What nonlinear transformation will we use?

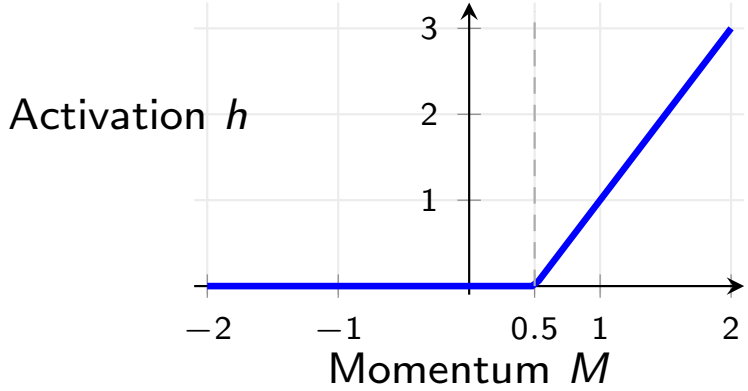
$$a = w^\top X + b, \qquad h = \text{ReLU}(a) = \max(0, a) = \begin{cases} 0, & a \leq 0, \\ a, & a > 0. \end{cases}$$

ReLU of the input score



Threshold: $a = 0$

ReLU of momentum: $a = 2M - 1$



Threshold: $M = 0.5$

Key distinction: ReLU always activates when its *input score* $a > 0$. But since $a = w^\top X + b$, the threshold in terms of the original features depends on the learned weights and bias.

Other Activation Functions

Q: Is ReLU the only useful choice?

Activation	**Characteristic**	**Tradeoff**
ReLU	$\max(0, a)$; simple, piecewise linear	Units may become inactive
Leaky ReLU	Small positive slope for $a < 0$	Requires choosing slope
Sigmoid	$1/(1 + e^{-a})$; smooth, bounded $(0, 1)$	Can saturate*
Tanh	Smooth, bounded $(-1, 1)$; zero-centered	Can saturate*

***Saturation:** For very large or small inputs, the activation function becomes nearly flat (slope near zero). This produces very small gradients, slowing or preventing effective learning through gradient descent.

We will use **ReLU** in our examples. For unrestricted signed return predictions, use a **linear final output**; the hidden layers can still be nonlinear.

A Numerical Forward Pass

Q: How does the network produce one return prediction?

Suppose the two **standardized** inputs are $M_i = 0.6$ and $V_i = 0.2$. A network with two ReLU units has these *illustrative* parameters:

$$a_{i1} = 1.0M_i - 1.0V_i + 0 = 0.4, \qquad h_{i1} = \max(0, 0.4) = 0.4,$$

$$a_{i2} = -0.5M_i + 1.0V_i + 0.2 = 0.1, \quad h_{i2} = \max(0, 0.1) = 0.1.$$

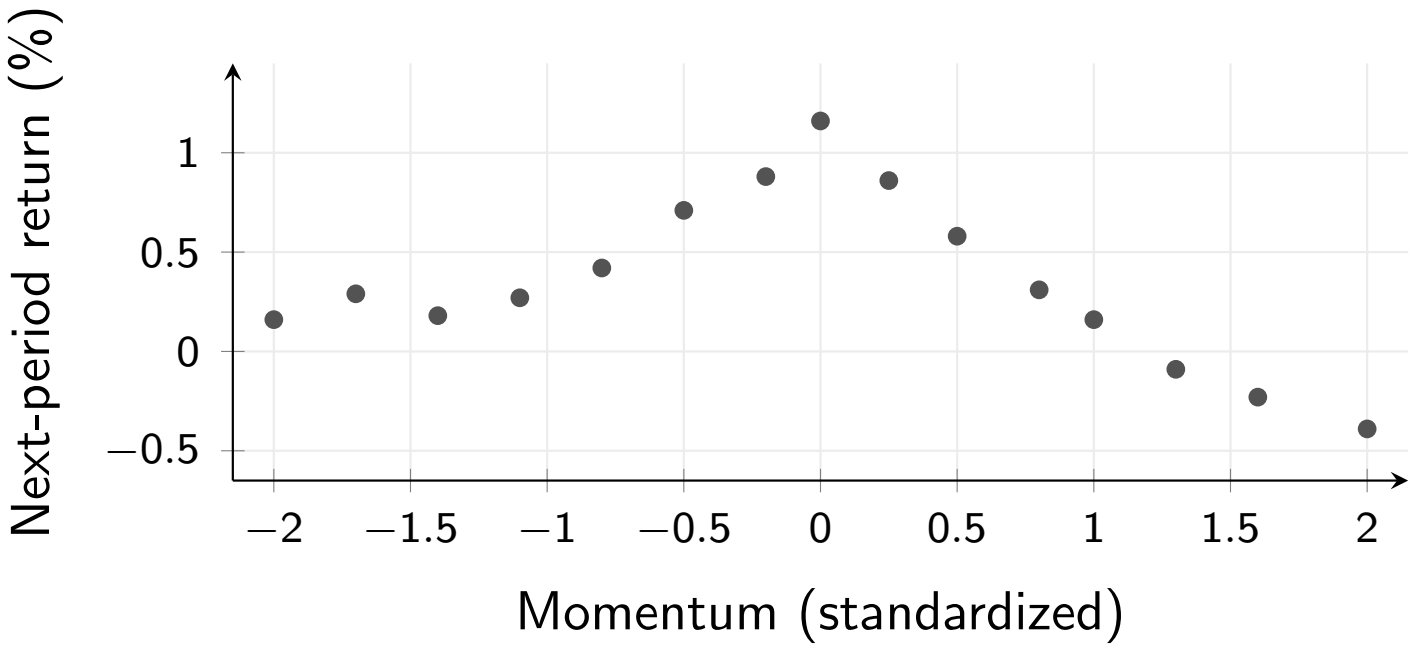
The output layer combines them:

$$\begin{aligned} \widehat{r}_i &= 0.002 + 0.010h_{i1} - 0.005h_{i2} \\ &= 0.002 + 0.010(0.4) - 0.005(0.1) \\ &= 0.0055 = \boxed{0.55\%}. \end{aligned}$$

This is **forward propagation**: applying the current weights to calculate a prediction. Training will change those weights.

Prediction Target

Suppose we're trying to models these returns

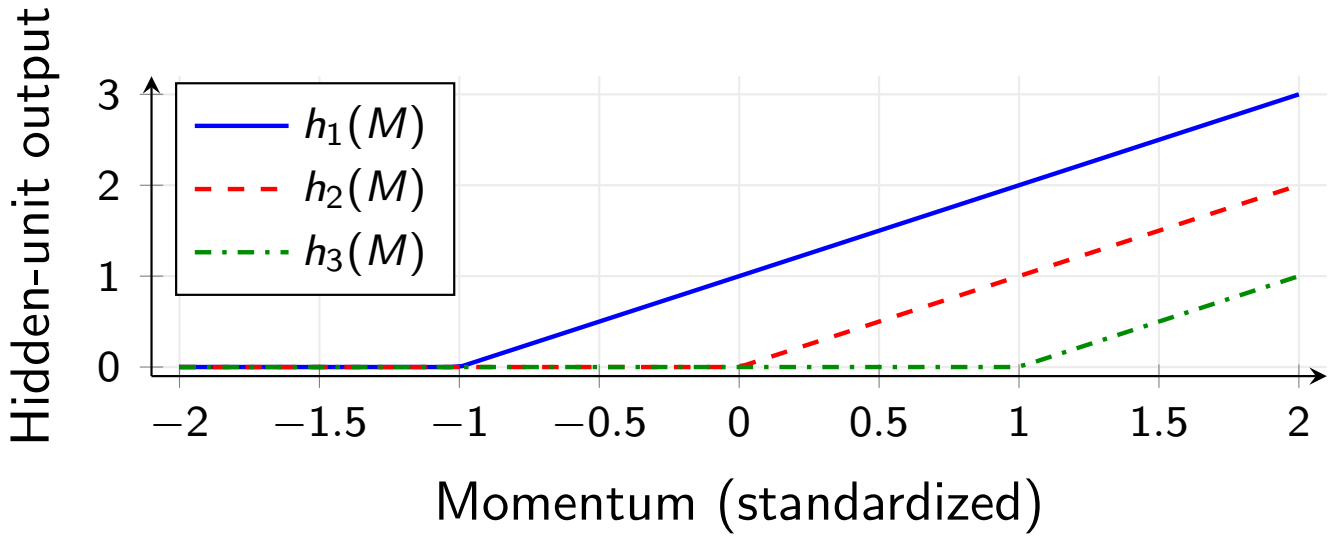


Learning Nonlinear Functions: Step 1

Q: What kinds of functions can individual ReLU units create?

Consider momentum alone, holding volatility fixed. Three illustrative units are

$$h_1(M) = \max(0, M + 1), \quad h_2(M) = \max(0, M), \quad h_3(M) = \max(0, M - 1).$$



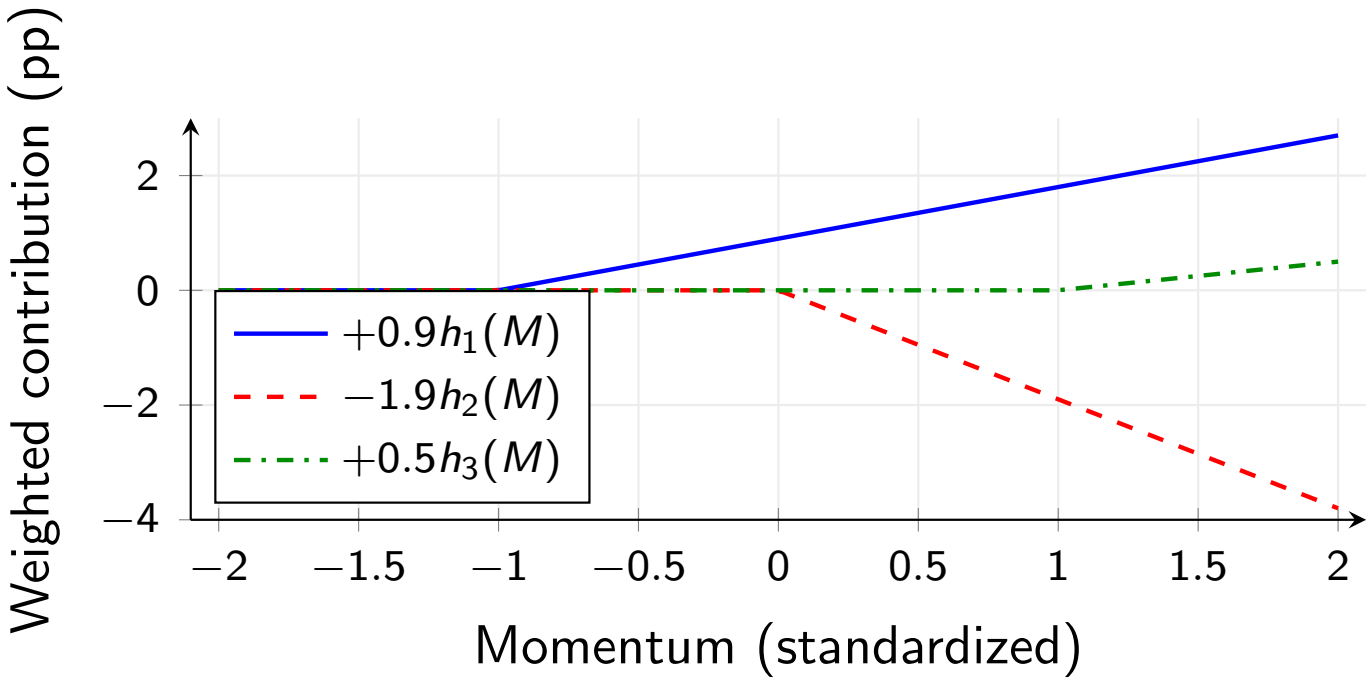
Unit	Activation threshold	Active when
$h_1(M)$	$M = -1$	$M > -1$
$h_2(M)$	$M = 0$	$M > 0$
$h_3(M)$	$M = 1$	$M > 1$

Learning Nonlinear Functions: Step 2

Q: How do simple hidden units combine into a richer function?

The output layer assigns a weight to each hidden unit. For illustration, measure predicted returns in **percentage points**:

$$\widehat{r}(M) = 0.2 + 0.9h_1(M) - 1.9h_2(M) + 0.5h_3(M).$$

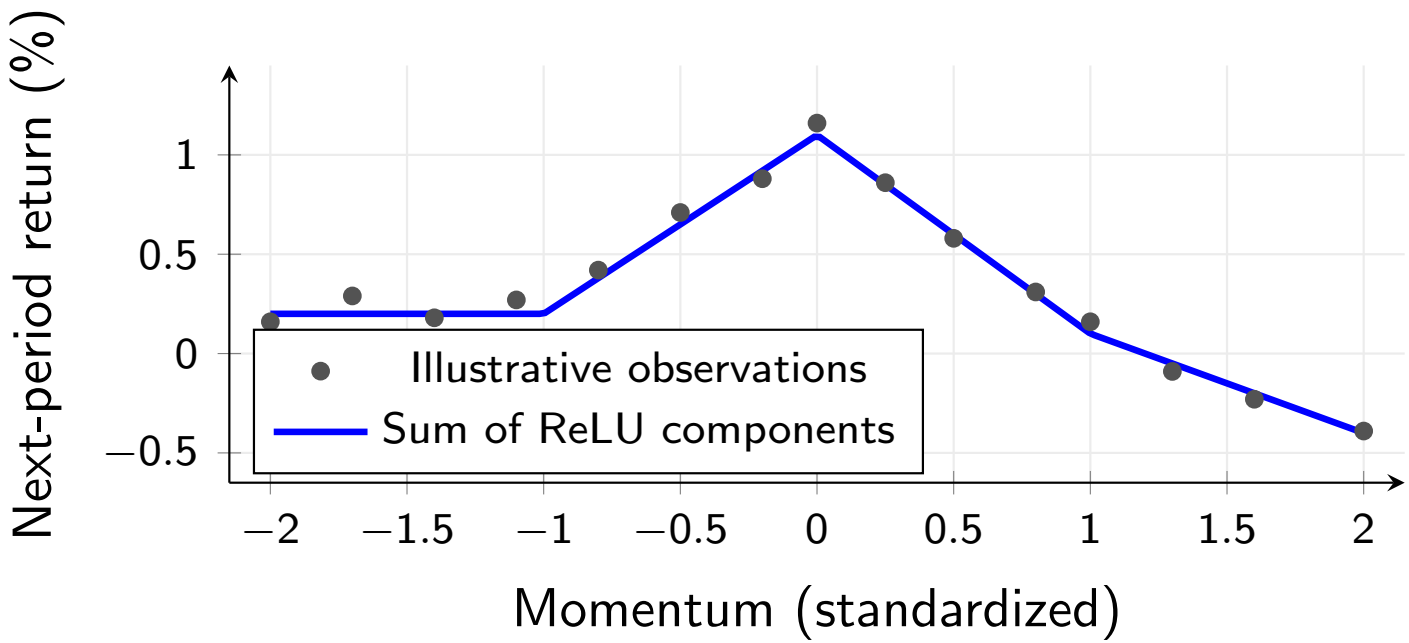


These are weighted *components*, not three separate return predictions.

Learning Nonlinear Functions: Step 3

Q: What does the sum of these components look like?

$$\widehat{r}(M) = 0.2 + 0.9\max(0, M+1) - 1.9\max(0, M) + 0.5\max(0, M-1).$$



The weights and breakpoints are illustrative; an actual network estimates them from training data.

The Connection to Splines

Q: Aren't these ReLU functions similar to spline basis functions?

Yes. With one input, a weighted sum of ReLU units creates a continuous **piecewise-linear** function. For

$$h_j(M) = \max(0, w_j M + b_j),$$

the bend occurs at $M = -b_j/w_j$ when $w_j \neq 0$.

- In a conventional spline-based GAM, we typically choose the basis and knot locations, then estimate coefficients.
- In a ReLU network, weights and biases also determine **where** bends occur; those values are estimated.
- With multiple inputs, hidden units transform **combinations** of characteristics, not just one feature at a time.

ReLU networks need not have the smooth joins of cubic splines. This flexibility can help or hurt.

Two Features: Momentum and Volatility

Q: How can a network learn interactions between features?

Consider a single hidden unit:

$$h_j(M, V) = \max(0, w_{jM} M + w_{jV} V + b_j)\,.$$

Its response to momentum depends on volatility because **both features enter before the activation**. For example, if $w_{jM} > 0$, this neuron becomes active when

$$M > \frac{-w_{jV} V - b_j}{w_{jM}}.$$

As volatility changes, the momentum threshold for activation changes. With many neurons, the network can learn complex interactions between momentum and volatility. No separate product term $M \times V$ is required as an explicit input, although we could add one.

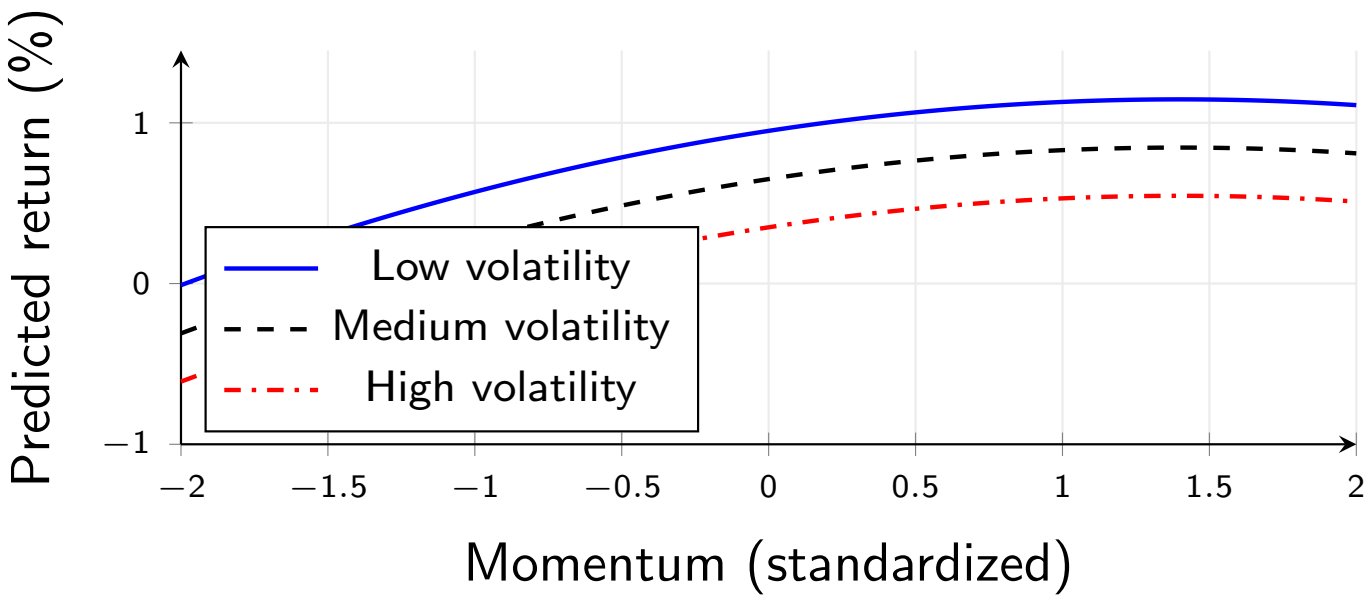
Additive GAM: Parallel Conditional Curves

Q: What does additivity imply for the effect of momentum?

With an additive GAM,

$$\widehat{r}(M, V) = \alpha + f_M(M) + f_V(V).$$

For any two volatility levels, the **vertical difference** between momentum curves is constant across M.

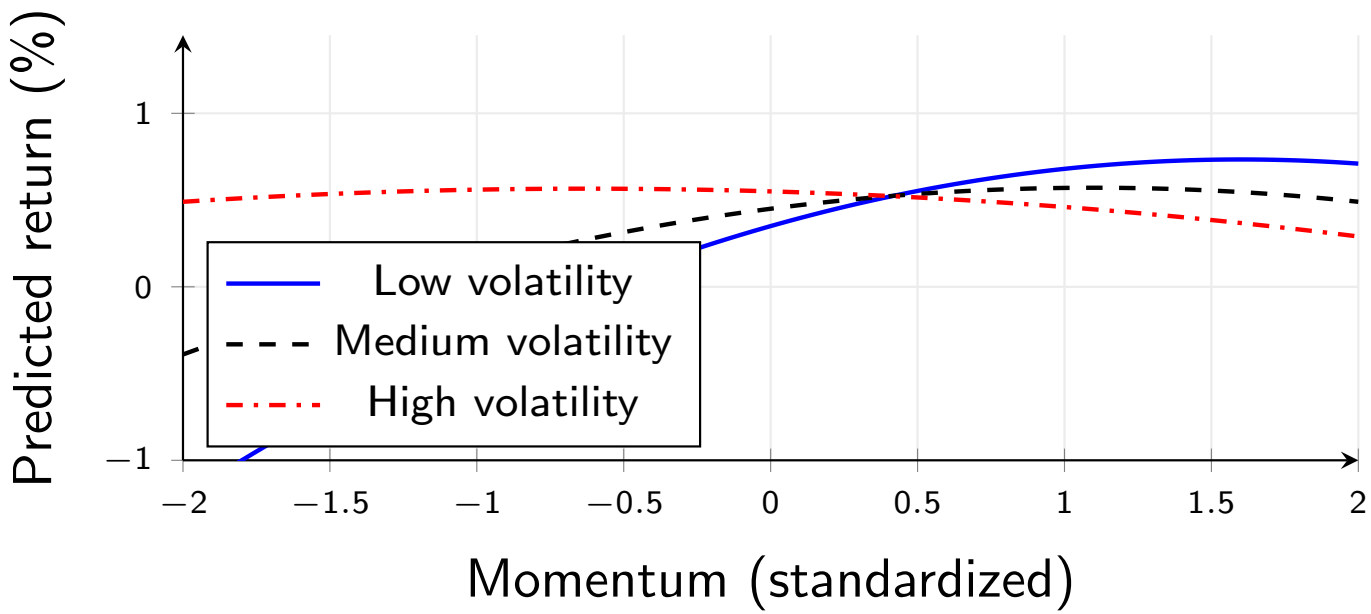


Illustrative curves for a standard *additive* GAM. A GAM with interactions need not have this restriction.

Neural Network: Changing Conditional Curves

Q: How could an interaction alter the momentum relationship?

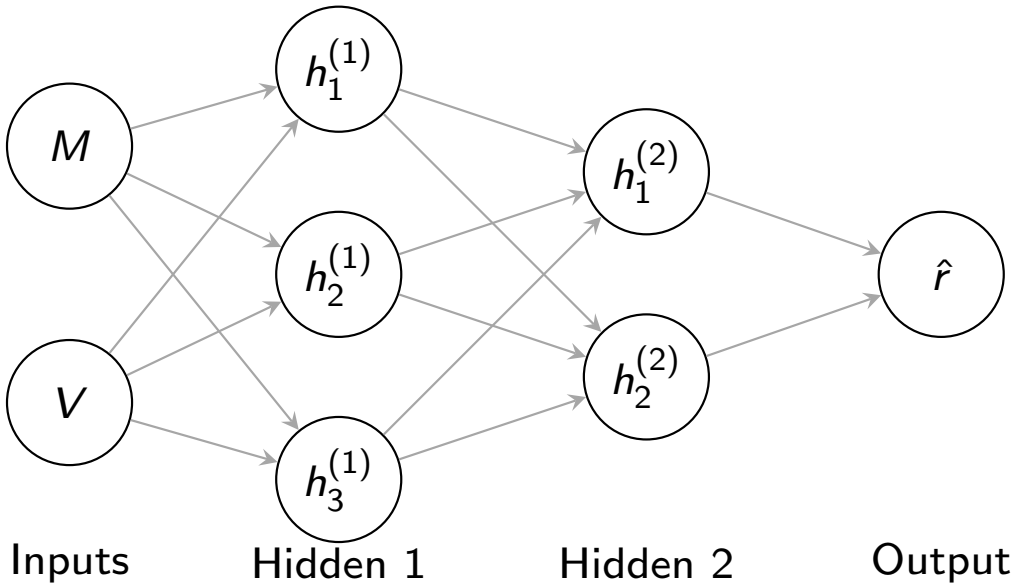
A neural network can learn that momentum has different slopes or curvature at different volatility levels.



Illustrative nonlinear interaction patterns, not fitted results from a particular network.

Adding More Hidden Layers

Q: What changes in a deeper network?



Each layer transforms the outputs of the previous layer:

$$h_i^{(\ell)} = \sigma\left(W^{(\ell)} h_i^{(\ell-1)} + b^{(\ell)}\right), \qquad h_i^{(0)} = X_i.$$

More depth can learn more complex **compositions** of features, but also adds parameters and overfitting risk.

Depth and Width

Q: What architectural choices do we make before training?

- **Width:** number of neurons in each hidden layer.
- **Depth:** number of hidden layers.
- **Activation:** ReLU, tanh, sigmoid, etc.
- **Output activation:** linear for unrestricted signed returns.

A useful starting point: a modest feedforward network, rather than assuming deeper or wider must be better.

This architecture is often called a *multilayer perceptron* (MLP). We are not using a recurrent or attention-based time-series model in this lecture.

The Tradeoff Between Width and Depth

Q: What do additional neurons and layers allow us to learn?

Width: More neurons within a hidden layer

- Learn more distinct nonlinear transformations in parallel.
- Capture finer variation across the feature space (e.g., different regions of momentum and volatility).
- Tradeoff: More parameters and greater potential to fit noise.

Depth: More successive hidden layers

- Each layer combines transformations learned by previous layers.
- Can represent increasingly complex relationships, including higher-order interactions.
- Tradeoff: Greater optimization complexity and potentially more overfitting.

Intuition: Width adds more nonlinear transformations; depth combines transformations across successive layers.

Linearity in Parameters

Q: Why is estimation different from OLS or a GAM?

Model	**What is estimated?**	**Linear in parameters?**
OLS with fixed transforms	Coefficients on specified features	Yes
Spline-based GAM	Coefficients on a specified basis	Yes*
Neural network	Hidden weights, biases, output weights	Generally no

For the network,

$$\widehat{r}_i = c + \sum_j v_j \, \mathrm{ReLU}(w_j^\top X_i + b_j),$$

parameters appear **inside** nonlinear transformations and **outside** them. *For a standard GAM with fixed basis, conditional on any smoothing parameters. More general GAM fitting procedures may be iterative.

Supervised Learning versus Autoencoders

Q: What is different from the networks we saw before?

	Autoencoder	**Return-prediction network**
Input	Characteristics X_i	Characteristics $X_{i,t}$
Output	Reconstruction $\widehat{X}_i$	Future return $\widehat{r}_{i,t+1}$
Learning objective	Reconstruct inputs	Predict observed returns
Learning type	Unsupervised	Supervised

Same building blocks: weights, biases, hidden layers, nonlinear activations, and numerical optimization.

Different objective: a supervised network learns features helpful for predicting returns; an autoencoder learns features helpful for reconstructing inputs.

The Neural Network Objective

Q: How are all the weights and biases estimated?

For each training observation, we know

$$(X_{i,t}, r_{i,t+1}), \quad X_{i,t} = (\mathrm{Mom}_{i,t}, \mathrm{Vol}_{i,t})^\top.$$

Collect all network parameters in θ. For continuous returns, a natural loss is **mean squared error**:

$$\min_\theta \ \mathcal{L}(\theta), \qquad \mathcal{L}(\theta) = \frac{1}{N_{\text{train}}} \sum_{(i,t)\in\mathcal{D}_{\text{train}}} [r_{i,t+1} - f(X_{i,t};\theta)]^2 .$$

- This is the same squared-error criterion used by OLS.
- But the fitted network is nonlinear in θ; generally there is no OLS-style closed-form solution.

Gradient Descent and Backpropagation

Q: How can the network minimize its loss?

Start with initial parameter values, then repeatedly update in a direction that reduces the loss:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_\theta \mathcal{L}(\theta^{(k)}).$$

- η: **learning rate**, which controls update size.
- $\nabla_\theta \mathcal{L}$: gradient of the loss with respect to all weights and biases.
- **Backpropagation**: efficient application of the chain rule to compute the gradient, working backward through the layers.
- In practice, updates often use **mini-batches** rather than the whole training dataset.

The optimization is generally **nonconvex**: different initializations or runs may lead to different solutions.

Choosing the Learning Rate

Q: What happens if the learning rate is too high or too low?

Recall the gradient descent update:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_\theta \mathcal{L}(\theta^{(k)}).$$

The learning rate η controls how far we move in the direction of the negative gradient.

- **Low learning rate:** Small parameter updates.
  - More cautious steps, but potentially very slow convergence.
- **High learning rate:** Large parameter updates.
  - Faster initial progress is possible.
  - But we may overshoot the minimum, oscillate, or diverge.

Tradeoff: We want updates large enough to learn efficiently, but small enough to converge reliably.

The learning rate is a hyperparameter chosen before training. It can remain fixed or be adjusted automatically during training according to a predetermined schedule or adaptive optimization algorithm.

The Training Loop

Q: What happens during one training epoch?

1. **Forward pass:** calculate predictions using current weights.
2. **Loss:** compare predictions with realized returns.
3. **Backpropagation:** calculate gradients of the loss.
4. **Update:** adjust weights and biases.

One **epoch** is a pass through the training dataset, usually via several mini-batches. **Financial-data reminder:** construct features using information

available at t; estimate scaling from training data only, and respect time ordering when forming validation and test samples.

Overfitting: Why We Need Restraint

Q: Why not use an extremely wide and deep network?

Networks may have far more adjustable parameters than OLS or a simple GAM. In finance, return-predictive relationships may be **weak, noisy, and unstable**.

- A flexible network can fit patterns specific to the training data.
- Lower training MSE does **not** imply better future predictions.
- Regularization, validation, and sensible architectural choices matter.

We keep performance evaluation brief here; the detailed treatment belongs in the separate model-evaluation lesson.

Regularization: Weight Decay

Q: How can we discourage an excessively complex fit?

Weight decay (L2 regularization) penalizes large network weights:

$$\min_{\theta} \left\{ \frac{1}{N_{\text{train}}} \sum_{(i,t)\in\mathcal{D}_{\text{train}}} [r_{i,t+1} - f(X_{i,t};\theta)]^2 + \lambda \sum_{w\in\mathcal{W}} w^2 \right\}.$$

- $\mathcal{W}$: the collection of penalized network weights (usually not biases).
- $\lambda \geq 0$: penalty strength.
- Higher λ favors smaller weights, possibly reducing overfitting at the expense of flexibility.

Other Ways to Limit Overfitting

Q: What else can we do beyond weight decay?

- **Dropout:** randomly deactivate hidden-unit outputs during training, discouraging reliance on particular units.
- **Early stopping:** stop training when validation loss ceases to improve.
- **Smaller architecture:** use fewer layers or units.
- **Time-respecting validation:** choose hyperparameters using later observations, without leaking future information into training.

Dropout is normally turned **off** when generating predictions.

Important Hyperparameters

Q: What does the analyst choose versus what the network learns?

Chosen by the analyst	**Learned during training**
Number of layers, units per layer	Hidden-layer weights and biases
Activation function	Output-layer weights and bias
Learning rate, batch size, epochs	—
Regularization strength, dropout	—

- Hyperparameters are chosen using validation data, not simply by minimizing training loss.
- Feature scaling helps training: standardize momentum and volatility using statistics estimated on the training set.

Interpreting a Return-Prediction Network

Q: Can we summarize the relationship with one coefficient?

Not generally. The estimated relationship between momentum and predicted returns can depend on **both momentum and volatility**:

$$\widehat{r}_{i,t+1} = f(\mathrm{Mom}_{i,t}, \mathrm{Vol}_{i,t}; \widehat{\theta}).$$

Unlike OLS, the network does not produce one momentum coefficient with a constant marginal effect. One simple descriptive tool is to plot **predicted returns versus momentum at different fixed volatility levels**, as in our earlier illustrations. Such response curves describe the model's predictions, not necessarily causal relationships.

From Predicted Returns to Trading Signals

Q: How do we convert predictions into a long–short signal?

At formation date t, use the trained network to produce $\widehat{r}_{i,t+1}$ using information available at t. Let $q_{.75,t}(\widehat{r})$ and $q_{.25,t}(\widehat{r})$ denote the **cross-sectional** 75th and 25th percentiles of predicted returns among eligible stocks at t. Define two **selection indicators**:

$$p_{i,t}^{(+)} = \mathbf{1}[\widehat{r}_{i,t+1} \geq q_{.75,t}(\widehat{r})]\,,$$

$$p_{i,t}^{(-)} = \mathbf{1}[\widehat{r}_{i,t+1} \leq q_{.25,t}(\widehat{r})]\,.$$

Here $p^{(+)}$ and $p^{(-)}$ are **0/1 indicators**, **not probabilities**.

Constructing the Long–Short Signal

Q: What is the final portfolio-selection rule?

Using the indicators just defined:

$$\boxed{s_{i,t} = p_{i,t}^{(+)} - p_{i,t}^{(-)}}$$

$$s_{i,t} = \begin{cases} +1, & \text{top quartile of predicted returns: long,} \\ -1, & \text{bottom quartile of predicted returns: short,} \\ 0, & \text{otherwise: neutral.} \end{cases}$$

- The **network** produces predicted returns.
- The **ranking rule** converts those predictions into a signal.

This selects long and short candidates, not their final dollar weights.

Other Neural Network Architectures

Q: Are all neural networks feedforward MLPs?

No. We have focused on a **feedforward multilayer perceptron** for cross-sectional return prediction. Other architectures include:

- **CNNs:** learn local patterns in structured inputs, including images and some financial sequences.
- **RNNs / LSTMs:** explicitly process sequences and can model temporal dependencies.
- **Transformers:** use attention to model relationships across positions in a sequence.
- **Autoencoders:** compress and reconstruct input features, as in our dimension-reduction lesson.

More complex architectures are not automatically better financial forecasters.

Neural Networks: Key Takeaways

Q: What should we remember for nonlinear regression?

1. Feedforward networks build return predictions from **learned nonlinear transformations** of observed characteristics.
2. Networks can learn **nonlinearity and interactions**; conventional additive GAMs need explicitly added interaction terms to do the latter.
3. Weights and biases are estimated from a **supervised loss**, using gradient-based optimization and backpropagation.
4. Flexibility raises **overfitting risk**; regularization and time-respecting validation matter.
5. The trained network produces $\widehat{r}_{i,t+1}$; the familiar $p^{(+)}$, $p^{(-)}$, and $s_{i,t}$ rule converts predictions to long–short candidates.

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
   - Regression Trees
   - Bagging
   - Random Forests
5. Gradient Boosting
6. Model Comparison

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
   - Regression Trees
   - Bagging
   - Random Forests
5. Gradient Boosting
6. Model Comparison

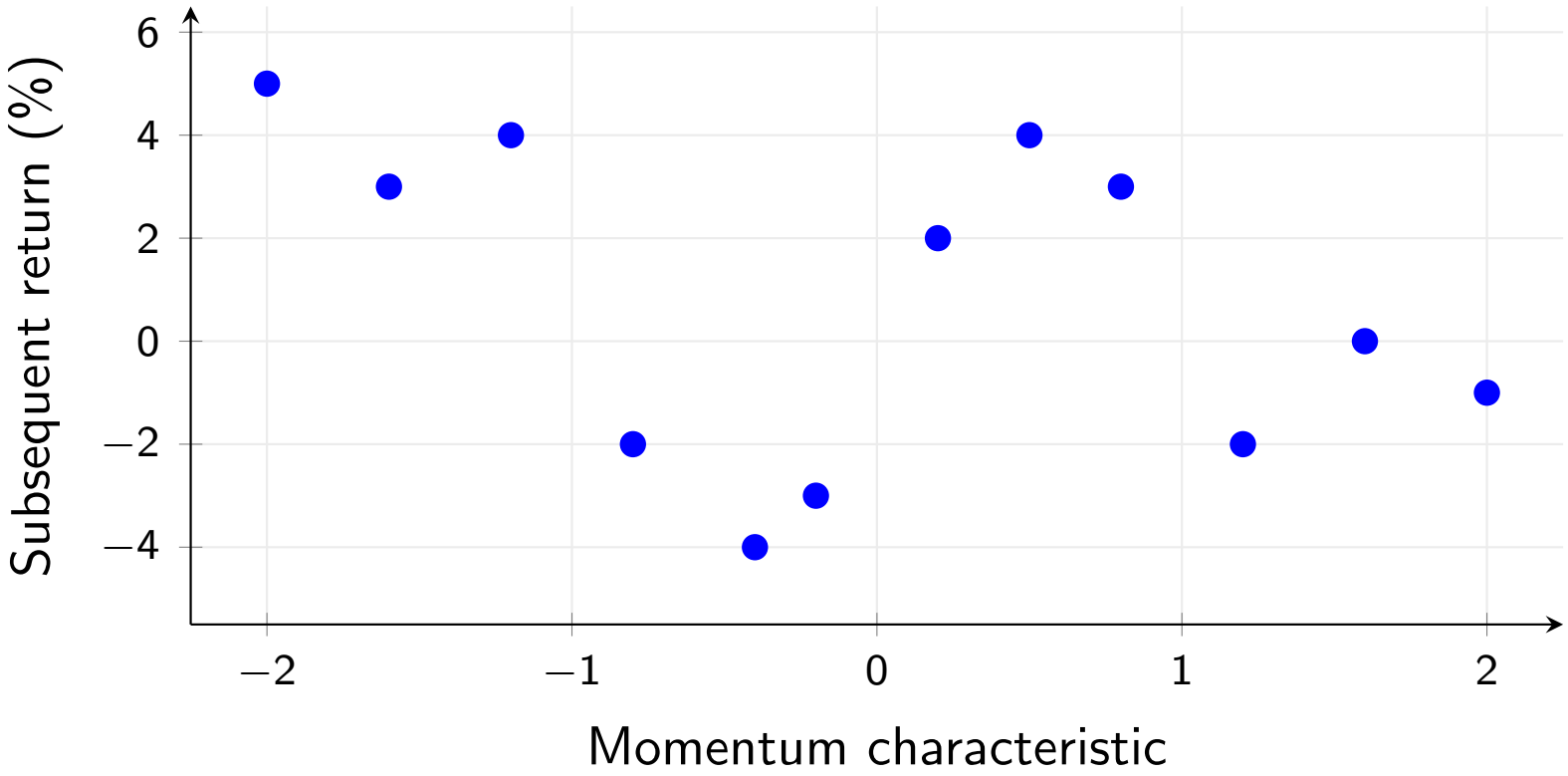
What if the Momentum–Return Relationship Is Nonlinear?

Suppose we want to explain **cross-sectional differences** in stock returns using momentum.

$$\mathbb{E}[r_i \mid X_i] = f(X_i), \qquad X_i = \text{momentum of stock } i.$$

- OLS requires us to specify the form of the relationship (e.g., linear or polynomial in momentum).
- GAMs let us construct flexible relationships using basis functions.
- **Regression trees** take another approach: divide the feature space into regions and predict a mean return in each region.

A Stylized Momentum–Return Relationship



Low and high momentum do not necessarily produce the lowest and highest returns: **reversal at the extremes** is possible.

Illustrative data only; the pattern is not an empirical claim about momentum.

From Classification Trees to Regression Trees

Recall: CART partitions the feature space using successive splitting rules.

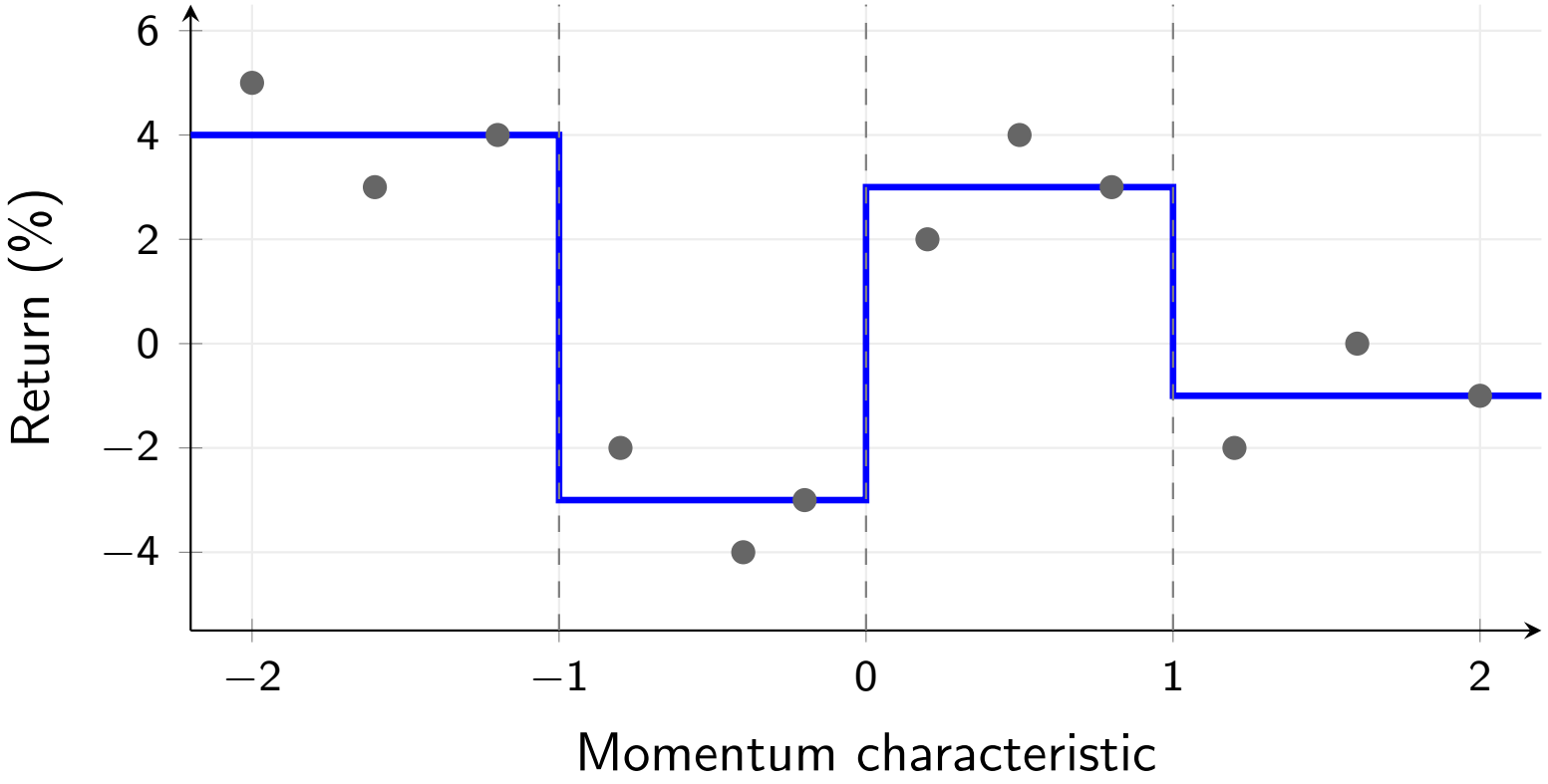
	Classification tree	**Regression tree**
Outcome	Class label	Continuous return
Leaf prediction	Class proportion / majority	Mean return
Splitting criterion	Impurity reduction (e.g., Gini)	Squared-error reduction

For a leaf (region) R_m,

$$\widehat{r}_m = \frac{1}{N_m} \sum_{i:X_i \in R_m} r_i.$$

Every stock assigned to that leaf receives the **same predicted return**.

A Regression Tree Is a Step-Function Approximation



The boundaries and leaf means are learned from the data; we did not specify a polynomial or smooth curve in advance.

Why Is the Leaf Prediction the Mean?

Within a leaf R, choose one constant prediction c for every stock.

$$\text{RSS}_R(c) = \sum_{i:X_i \in R} (r_i - c)^2.$$

Q: Which value of c minimizes squared error? Differentiate and set the derivative equal to zero:

$$\frac{d\ \text{RSS}_R(c)}{dc} = -2 \sum_{i:X_i \in R} (r_i - c) = 0.$$

Thus

$$\boxed{\widehat{r}_R = c^* = \frac{1}{N_R} \sum_{i:X_i \in R} r_i.}$$

A: The mean is the least-squares prediction within each leaf, just as in an intercept-only OLS regression.

How Does CART Choose a Regression Split?

At a node with observations R, consider a momentum threshold s:

$$R_L(s) = \{i \in R : X_i \leq s\}, \qquad R_R(s) = \{i \in R : X_i > s\}.$$

For each candidate s, predict the mean in each child node and calculate

$$\mathrm{RSS}_{\mathrm{split}}(s) = \sum_{i \in R_L(s)} (r_i - \bar{r}_L)^2 + \sum_{i \in R_R(s)} (r_i - \bar{r}_R)^2.$$

Choose the split that maximizes the improvement:

$$\boxed{\Delta\,\mathrm{RSS}(s) = \mathrm{RSS}_{\mathrm{parent}} - \mathrm{RSS}_{\mathrm{split}}(s).}$$

With multiple features, repeat the comparison for each candidate **feature and threshold** (j, s).

Worked Example: Twelve Stocks

Suppose these returns are observed across twelve stocks in one cross section.

Stock i	1	2	3	4	5	6
Momentum X_i	-2.0	-1.6	-1.2	-0.8	-0.4	-0.2
Return r_i (%)	5	3	4	-2	-4	-3

Stock i	7	8	9	10	11	12
Momentum X_i	0.2	0.5	0.8	1.2	1.6	2.0
Return r_i (%)	2	4	3	-2	0	-1

Before splitting, the prediction for every stock is the sample mean

$$\hat{r}_i = \bar{r}_{parent} = 0.75\%, \qquad \mathrm{RSS}_{\mathrm{parent}} = 106.25 \text{ (percentage-points squared)}.$$

Worked Example: Calculate RSS for $s = -1$

The threshold $s = -1$ separates three stocks on the left and nine on the right.
Left child: returns $\{5, 3, 4\}$, so $\bar{r}_L = 4$ and

$$\mathrm{RSS}_L = (5-4)^2 + (3-4)^2 + (4-4)^2 = \boxed{2}.$$

Right child: returns $\{-2, -4, -3, 2, 4, 3, -2, 0, -1\}$, so $\bar{r}_R = -1/3$. Use the computational shortcut $\mathrm{RSS} = \sum r_i^2 - n\bar{r}^2$:

$$\mathrm{RSS}_R = 63 - 9(-1/3)^2 = \boxed{62}.$$

Thus

$$\mathrm{RSS}_{\mathrm{split}} = 2 + 62 = 64, \qquad \Delta\,\mathrm{RSS} = 106.25 - 64 = \boxed{42.25}.$$

Worked Example: Compare Candidate Thresholds

Q: Where should we make the first split on momentum?

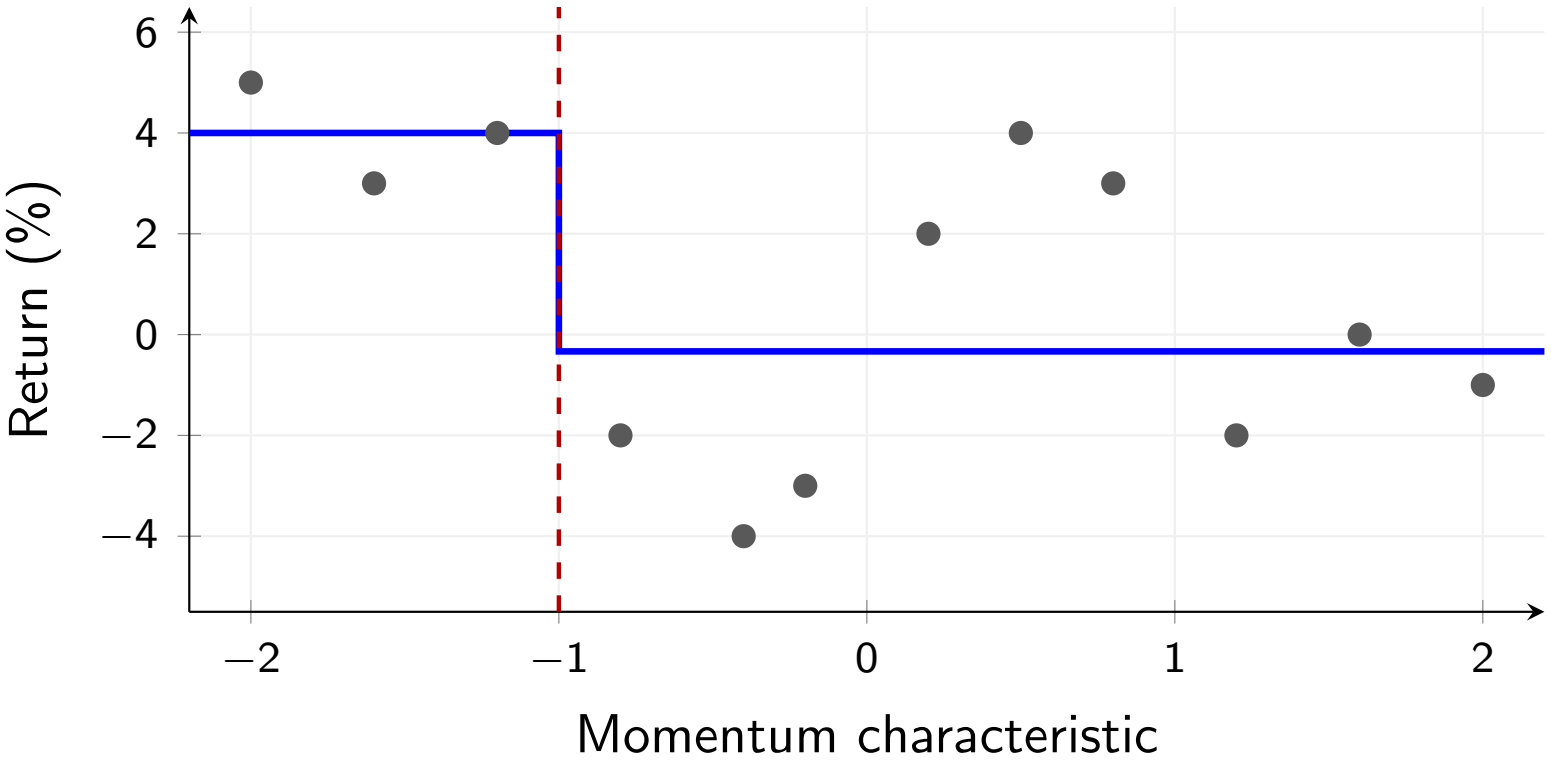
Threshold s	$\bar{r}_L$ (%)	$\bar{r}_R$ (%)	RSS after split	RSS reduction
-1	4.00	-0.33	64.00	**42.25**
0	0.50	1.00	105.50	0.75
1	1.33	-1.00	94.00	12.25

A: Split at $s = -1$, which produces the largest reduction in RSS.

$$\underbrace{106.25}_{\text{RSS before}} - \underbrace{[2.00 + 62.00]}_{\text{RSS after}} = \boxed{42.25}.$$

Actual CART tests thresholds between *adjacent sorted, distinct* feature values. We show three candidates for simplicity; -1 is also best among all candidates for this dataset.

The First Split: Two Regions



$X_i \leq -1$: predict 4.00%. $X_i > -1$: predict −0.33%.

Grow the Tree Recursively

Once we split at $X = -1$, we repeat the same procedure **within each resulting node**.

- The left node ($X \leq -1$) has mean return 4% and RSS 2.
- For the right node ($X > -1$), another useful split is $X = 0$:

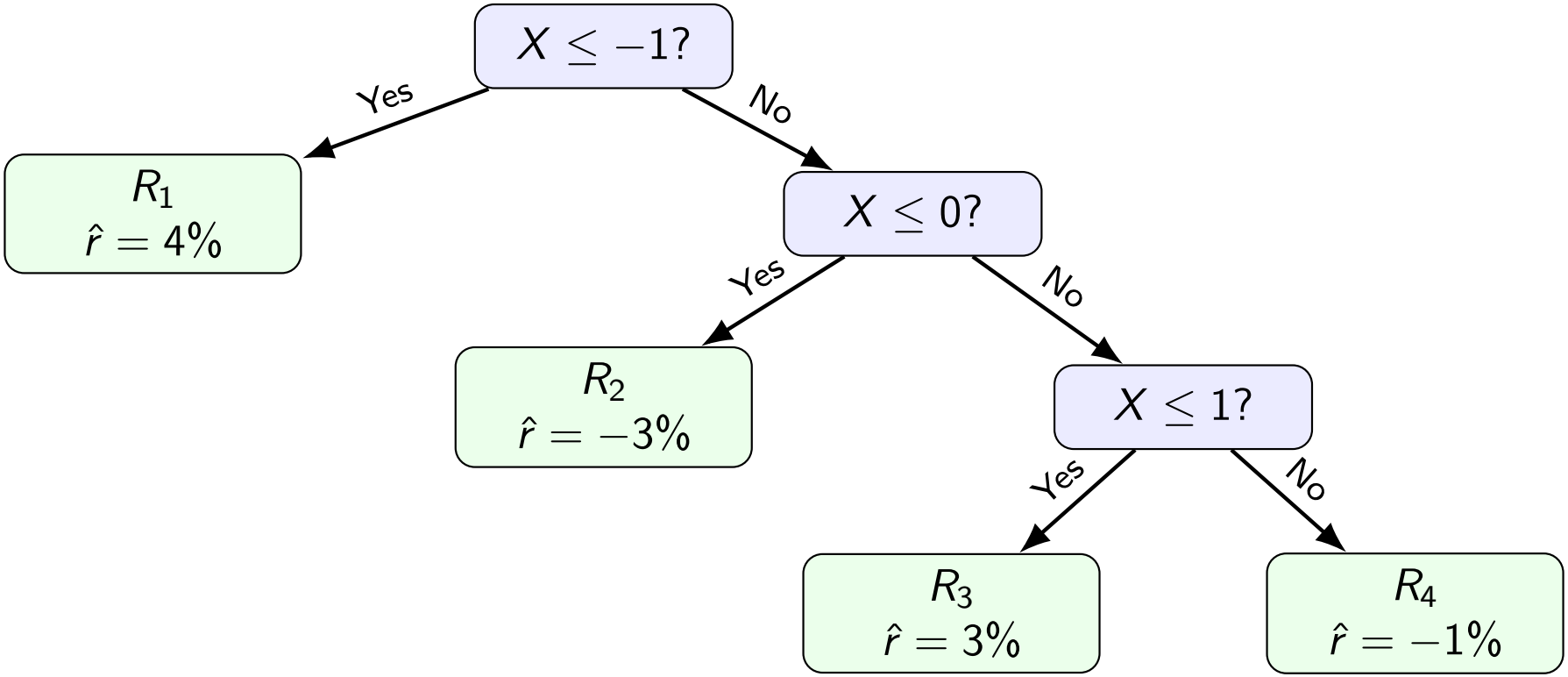
$$\begin{aligned} -1 < X \leq 0: &\quad \bar{r} = -3\%, \quad \mathrm{RSS} = 2, \\ X > 0: &\quad \bar{r} = 1\%, \quad \mathrm{RSS} = 28. \end{aligned}$$

- Splitting the final group at $X = 1$ produces two more leaves with means 3% and -1%.

Total training RSS falls from 106.25 (no split) to 8 (four leaves). **But:** Lower training RSS does not necessarily mean better predictions for new stocks.

Equivalent Tree Representation

Each path through the tree identifies one momentum region.



The tree represents the same four regions shown in the step-function plot.

What Is the Fitted Regression Function?

Let the tree create J terminal regions $R_1, \ldots, R_J$.

$$\widehat{f}(X_i) = \sum_{m=1}^{J} \bar{r}_m \, \mathbf{1}\{X_i \in R_m\}.$$

- The model is **nonlinear in momentum**, because a stock can cross a threshold and receive a different prediction.
- It is **piecewise constant**, with abrupt jumps at region boundaries.
- In more than one feature, each split selects a feature and threshold; the final regions become rectangles (or higher-dimensional boxes).

Q: Must these abrupt jumps be economically realistic?
A: No. They are a consequence of the tree approximation, and can make individual-tree predictions unstable.

Controlling the Complexity of a Tree

A tree that keeps splitting can achieve very low **training** RSS, including by fitting noise. We can try to mitigate via pre or post pruning.

Pre-pruning (stop early):

- Limit maximum depth.
- Require a minimum number of observations per leaf.
- Require a minimum improvement in squared error.

Post-pruning (grow, then cut back):

$$\min_{T} \{\text{RSS}(T) + \alpha|T|\},$$

where $|T|$ is the number of terminal leaves and α penalizes complexity. Larger α favors simpler trees; select complexity using validation data.

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
   - Regression Trees
   - Bagging
   - Random Forests
5. Gradient Boosting
6. Model Comparison

Why Not Rely on One Regression Tree?

A single tree has two related weaknesses:

- **Instability / high variance:** a few changed observations can move an early split and alter many predictions.
- **Overfitting:** deep trees may explain idiosyncratic return noise rather than a durable cross-sectional pattern.

Idea: Instead of trusting one tree, fit *many* trees to perturbed samples and average their predictions.

$$\widehat{f}_{\text{ensemble}}(X_i) = \frac{1}{M} \sum_{m=1}^{M} T_m(X_i).$$

This is the intuition behind **bootstrap aggregation (bagging)**.

Bagging: Where Do Different Trees Come From?

Goal: Predict the expected return for a stock with features X_i.

We start with a cross-sectional training sample of N stocks. For $m = 1, \ldots, M$:

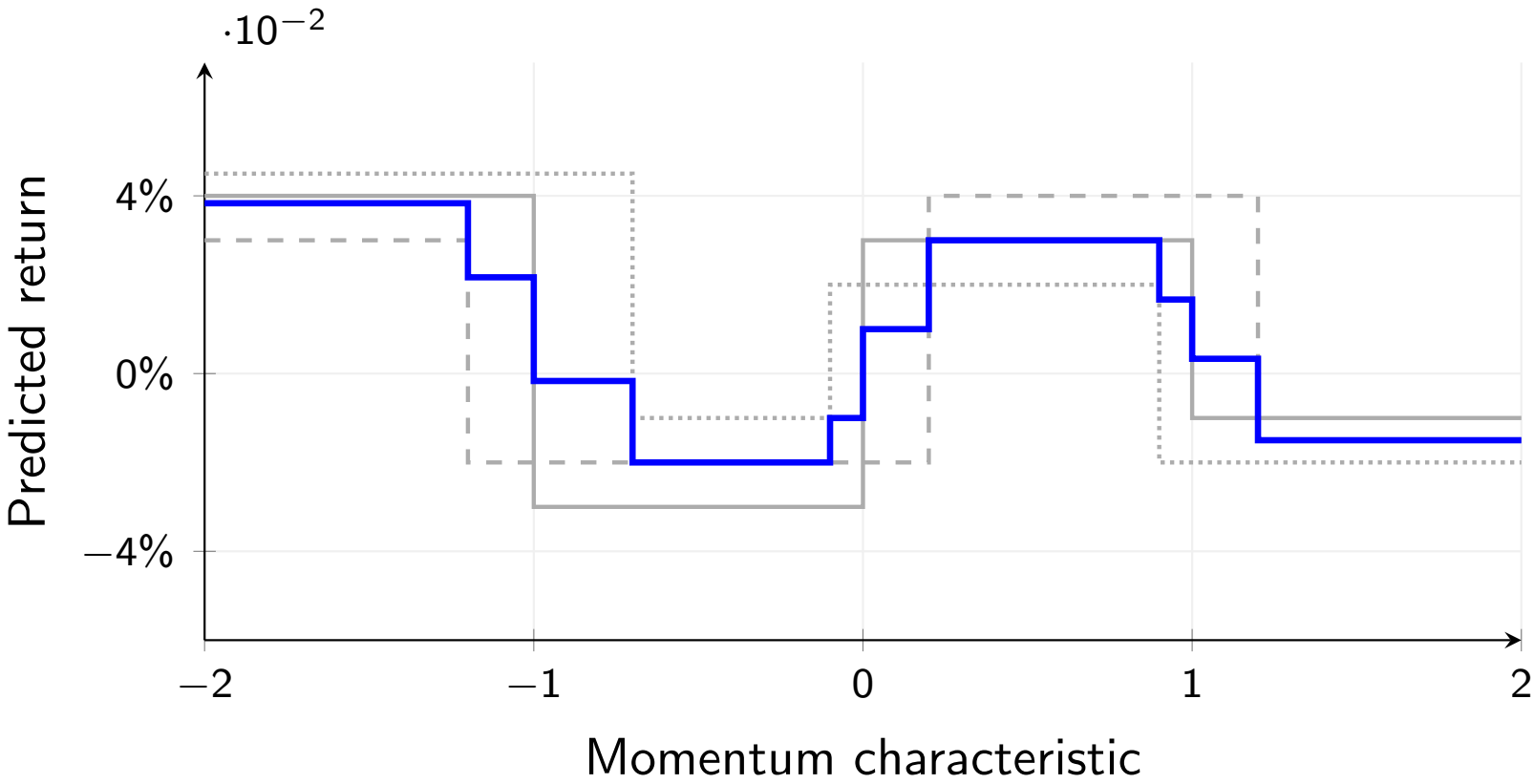
1. Draw N observations **with replacement** from the training sample (a bootstrap sample).
2. Fit a regression tree to that sample.
3. Retain its prediction $T_m(X_i)$ for any stock with features X_i.

Because observations can be repeated or omitted, the trees have different training samples and can have different splits and leaf means.

Prediction:

$$\widehat{f}_{\text{bag}}(X_i) = \frac{1}{M} \sum_{m=1}^{M} T_m(X_i).$$

Averaging Trees: What Changes?



Gray: individual trees. Blue: their pointwise average.

Averaging reduces sensitivity to any one tree's splits. The forest need not be perfectly smooth, but it typically has many smaller steps.

Why Does Averaging Reduce Variance?

Suppose, at a fixed feature vector X, each tree prediction $T_m(X)$ has variance σ^2, and every pair of trees has correlation $0 \leq \rho \leq 1$. For $\bar{T}(X) = M^{-1} \sum_{m=1}^{M} T_m(X)$,

$$\mathrm{Var}(\bar{T}) = \frac{1}{M^2} \left(\sum_{m=1}^{M} \mathrm{Var}(T_m) + \sum_{m \neq \ell} \mathrm{Cov}(T_m, T_\ell) \right).$$

Substitute $\mathrm{Var}(T_m) = \sigma^2$ and $\mathrm{Cov}(T_m, T_\ell) = \rho\sigma^2$:

$$= \frac{1}{M^2} \left[M\sigma^2 + M(M-1)\rho\sigma^2 \right].$$

$$\boxed{\mathrm{Var}(\bar{T}) = \sigma^2 \left(\rho + \frac{1-\rho}{M} \right).}$$

Illustrative equal-variance, equal-correlation assumption; variance refers to the variability of tree predictions across training samples/randomization.

Why Tree Correlation Matters

With $M = 100$ trees, the variance formula becomes

$$\mathrm{Var}(\bar{T}) = \sigma^2 \left(\rho + \frac{1-\rho}{100} \right).$$

Tree correlation ρ	Ensemble variance	Interpretation
0.80	$0.802\,\sigma^2$	Little diversification
0.20	$0.208\,\sigma^2$	Much more diversification
0	$0.010\,\sigma^2$	Independent-tree benchmark

As $M \to \infty$ (for fixed $\rho \geq 0$),

$$\mathrm{Var}(\bar{T}) \longrightarrow \rho\sigma^2.$$

Q: Why is adding more trees not enough? **A:** Correlation creates a variance floor. We also want **less correlated trees**, without making each tree too weak.

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
   - Regression Trees
   - Bagging
   - Random Forests
5. Gradient Boosting
6. Model Comparison

Why Can Bagged Trees Remain Highly Correlated?

- If 12–2 momentum is a particularly strong predictor, many trees may select it for their first split, despite using different bootstrap samples.
- Because the samples come from the same underlying data, the selected split thresholds may also be similar.
- The resulting groups of stocks may therefore be similar, producing similar predictions and correlated prediction errors.

Bagging randomizes the *observations*, but at every split each tree may still search over *all* available features.
Random forest: Randomize the features available **at each split**, as well as the bootstrap sample.

Random Features Encourage Different Trees

Suppose the cross-sectional feature vector contains

$$X_i = (\text{long-horizon momentum},\ \text{short-horizon momentum},\ \text{volatility},\ \text{turnover}).$$

- Bagging: at each node, search all $K = 4$ features for the split with the largest RSS reduction.
- Random forest (example): randomly choose only $m_{\text{try}} = 2$ of the $K = 4$ features **at that node**.
- Search for the best split **among those two features only**.
- Draw a fresh random subset at the **next node**.

This may make individual trees somewhat weaker, but can reduce their correlation and improve the ensemble.

Random Forest Regression: The Algorithm

Given training data $\{(X_i, r_i)\}_{i=1}^{N}$, construct M trees. For $m = 1, \ldots, M$:

1. Draw a bootstrap sample of the training observations.
2. At each node, randomly choose m_{try} candidate features.
3. Among those features and their candidate thresholds, choose the split with the largest reduction in RSS.
4. Continue until the tree meets its stopping rule (often producing deep trees).

For a new stock with characteristics X_i, average predictions:

$$\widehat{r}_i = \widehat{f}_{\text{RF}}(X_i) = \frac{1}{M} \sum_{m=1}^{M} T_m(X_i).$$

Bagging vs. Random Forests

	Bagging	**Random forest**
Bootstrap observations	Yes	Yes
Random features per split	No	Yes
Each leaf predicts	Mean return	Mean return
Combine trees by	Averaging	Averaging
Main purpose	Reduce variance	Seek less correlated trees

A random forest is **bagging plus random feature selection at each split**. If there is only **one** available feature, feature randomization cannot add diversity: the distinction from bagging essentially disappears.

Random Forest Hyperparameters

Hyperparameter	**Why it matters**
Number of trees M	More trees stabilize the average; gains eventually level off, while computation grows.
Features per split m_{try}	Smaller values can reduce correlation, but may weaken each tree.
Minimum samples per leaf	Larger leaves smooth predictions and reduce the ability to fit noise.
Maximum tree depth	Limits how finely each tree partitions the feature space.

The objective is not to minimize training RSS. Hyperparameters govern the tradeoff between the flexibility of each tree and the stability of the ensemble. We select hyperparameters to minimize prediction error using validation data (or out-of-bag predictions). In finance, validation should respect the time structure of the data.

Regression Trees and Random Forests: Summary

	Key mechanism	**Main consideration**
Single regression tree	Split feature space; predict a mean return within each leaf	Interpretable, but potentially unstable and prone to overfitting
Bagging	Bootstrap trees; average predictions	Reduces variance, but trees can remain correlated
Random forest	Bagging plus random features at each split	Can lower tree correlation and improve ensemble stability

OLS and GAMs provide coefficients on specified terms or basis functions. Trees and forests instead encode relationships through splitting rules.

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
5. Gradient Boosting
6. Model Comparison

Motivation: Improving a Prediction in Stages

Recall: a regression tree predicts a mean return within each leaf.
New idea: instead of building one large tree, build a sequence of small trees that correct the current predictions.
We will use the *same 12 hypothetical momentum–return observations* from the Regression Trees section.

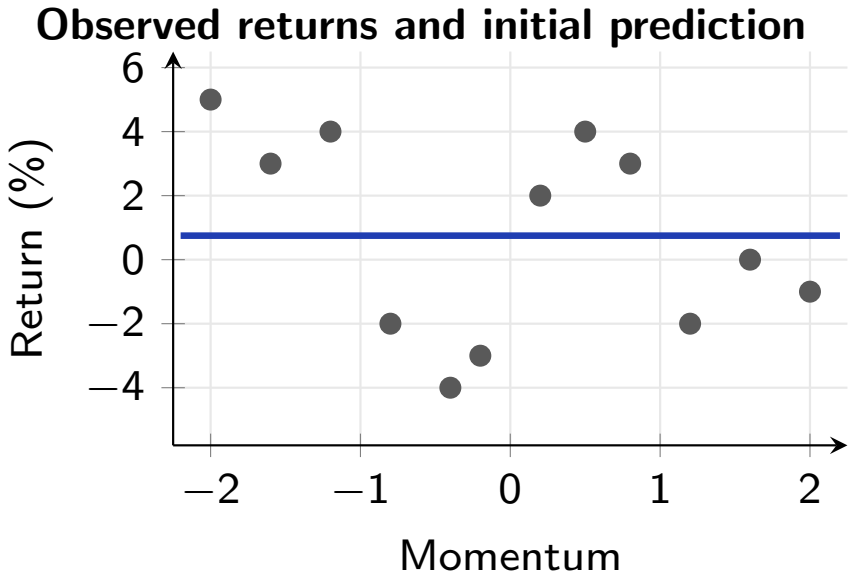
- Start with a constant prediction for every stock.
- Fit a small tree to the remaining prediction errors.
- Add part of that tree's correction, then repeat.

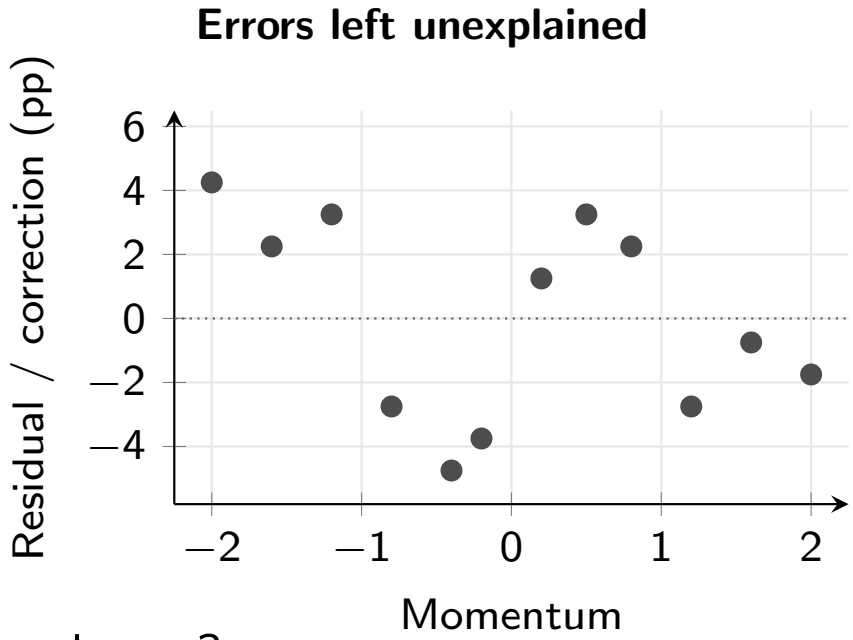
Q: Why not add the entire correction?
A: Prediction errors contain both systematic patterns and noise. Smaller corrections can reduce the risk of overfitting to the noise.

Step 0: Begin with the Mean Return

For squared-error regression, the best *constant* prediction is the sample mean:

$$F_0(X_i) = \bar{r} = 0.75\%.$$





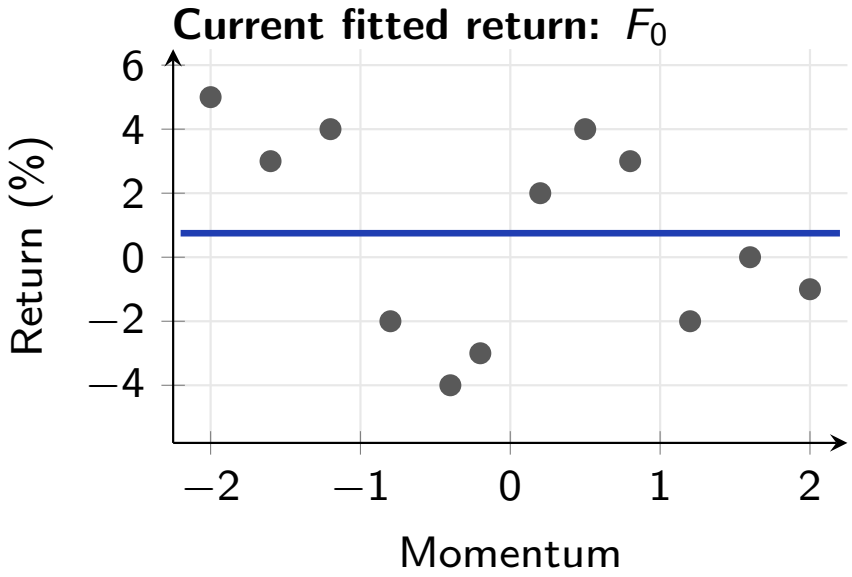
Q: Are these errors just random scatter around zero?

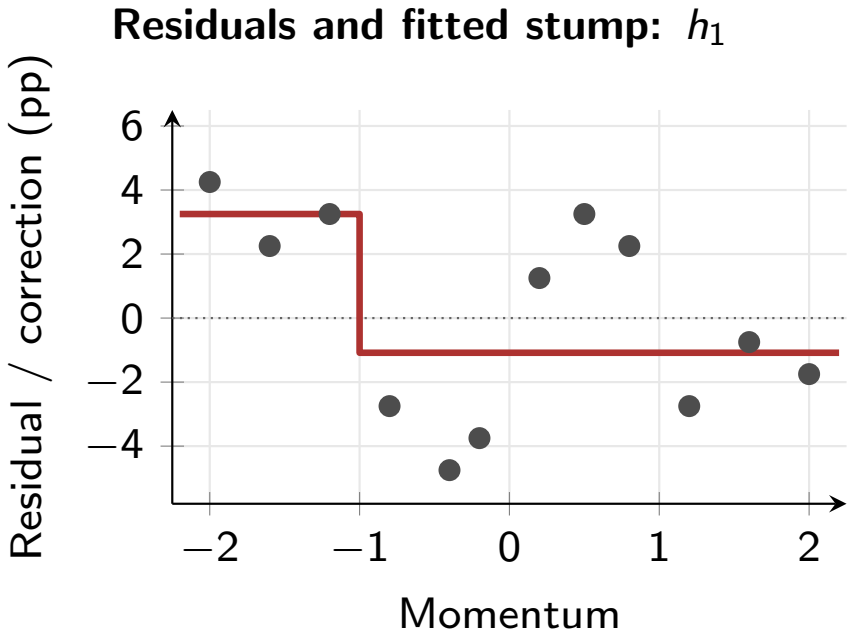
A: No. The signs and sizes vary systematically across momentum. A tree may capture part of that remaining pattern.

Step 1: Fit a Tree to the Residuals

Initial residual for each stock:

$$e_i^{(0)} = r_i - F_0(X_i).$$





Q: What does the new tree predict?

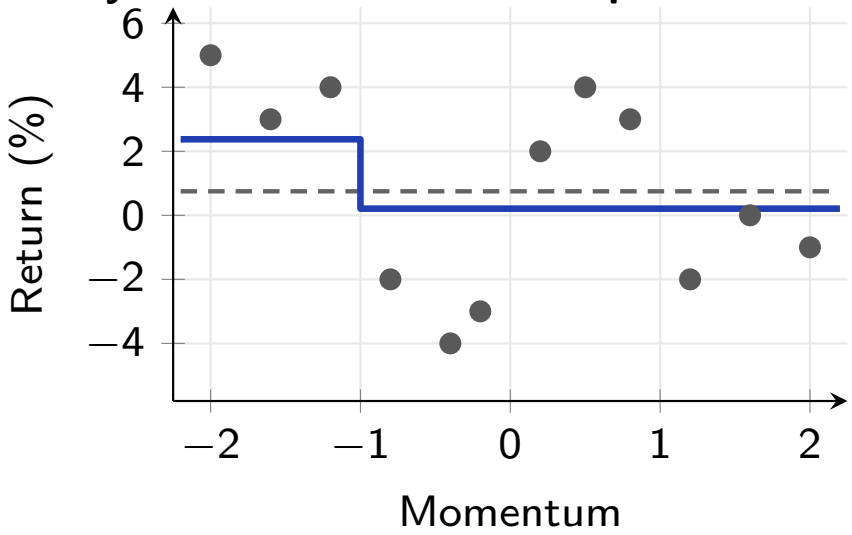
A: Mean residual within each leaf. Its best split is $X = -1$: $h_1 = +3.25$ pp on the left, and $h_1 = -1.08$ pp on the right.

Step 1: Add Only Part of the Correction

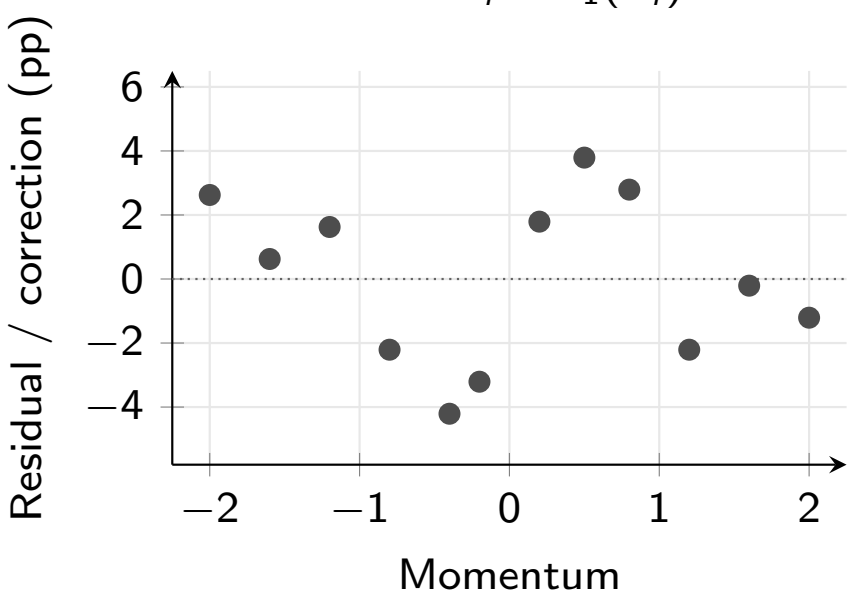
With learning rate $\nu = 0.5$, update the prediction:

$$F_1(X) = F_0(X) + 0.5h_1(X).$$

Gray dashed: old fit; blue: updated fit



New residuals: $r_i - F_1(X_i)$

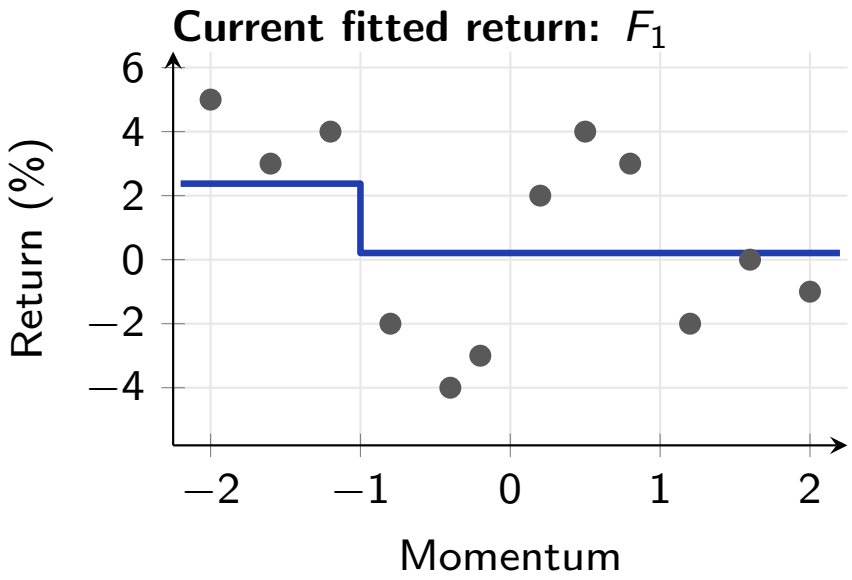


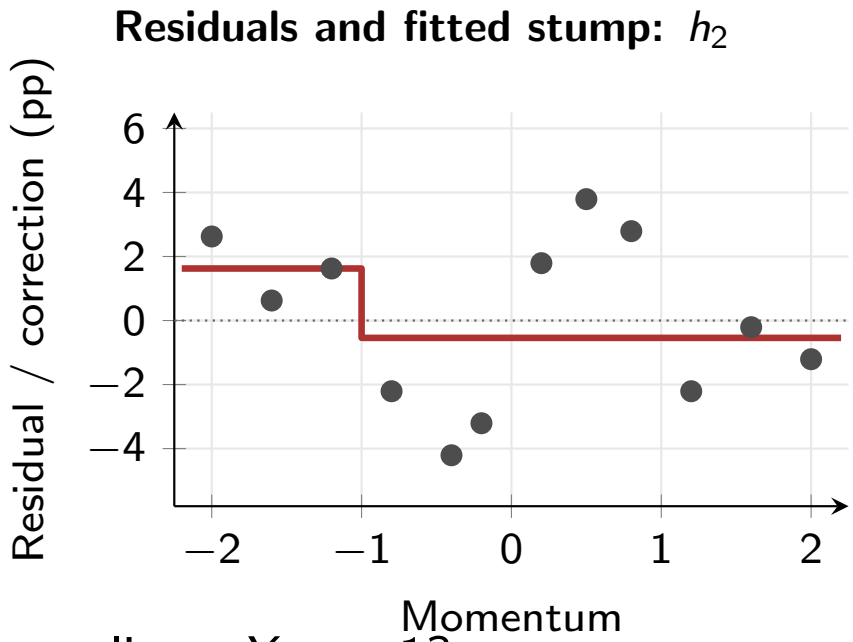
Q: Why do systematic errors remain?

A: We applied only half the fitted correction, and a one-split tree cannot capture every pattern.

Step 2: Fit Another Tree to the New Residuals

We now fit a new stump to $e_i^{(1)} = r_i - F_1(X_i)$.



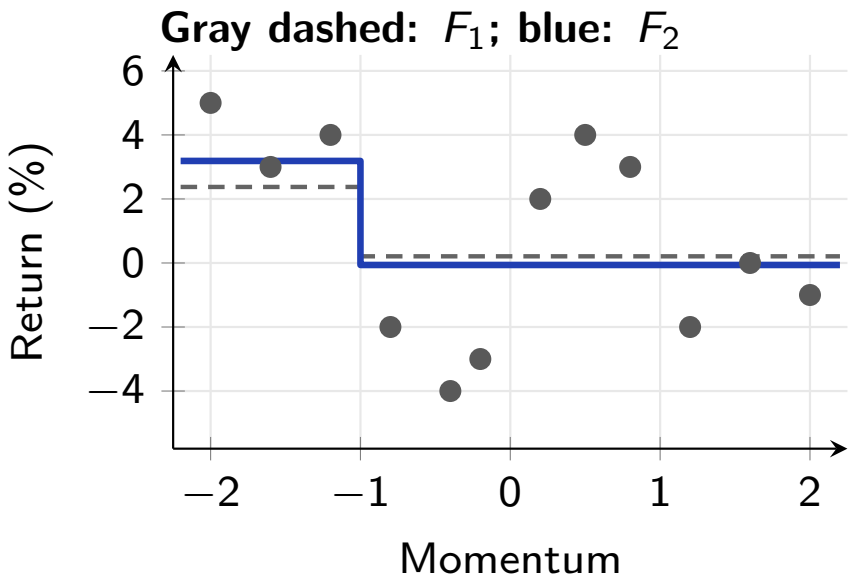


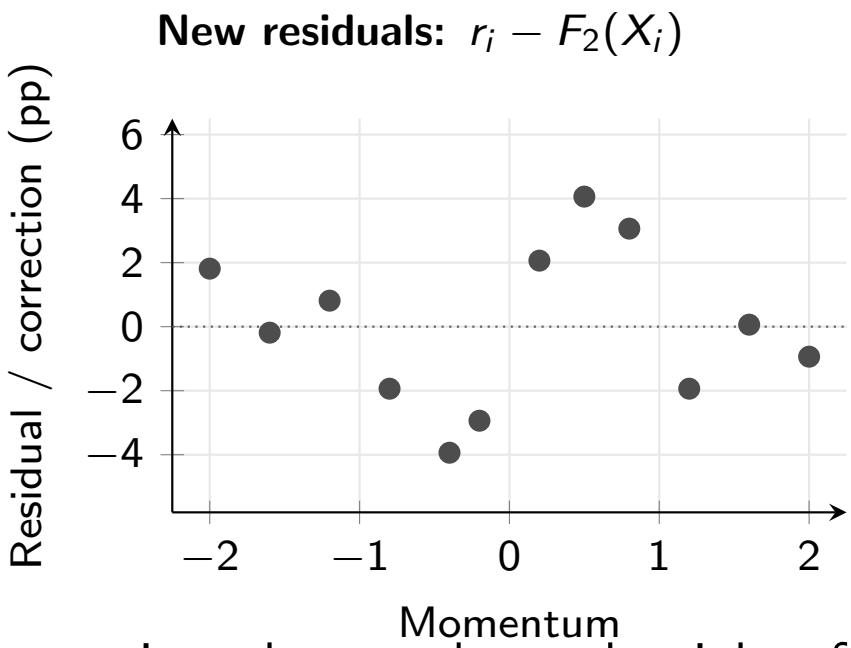
Q: Why did the second tree choose the *same* split at $X = -1$?

A: Because we applied only half the first tree's correction, some of the same systematic pattern remains in the residuals.

Step 2: Apply Another Partial Correction

$$F_2(X) = F_1(X) + 0.5h_2(X).$$

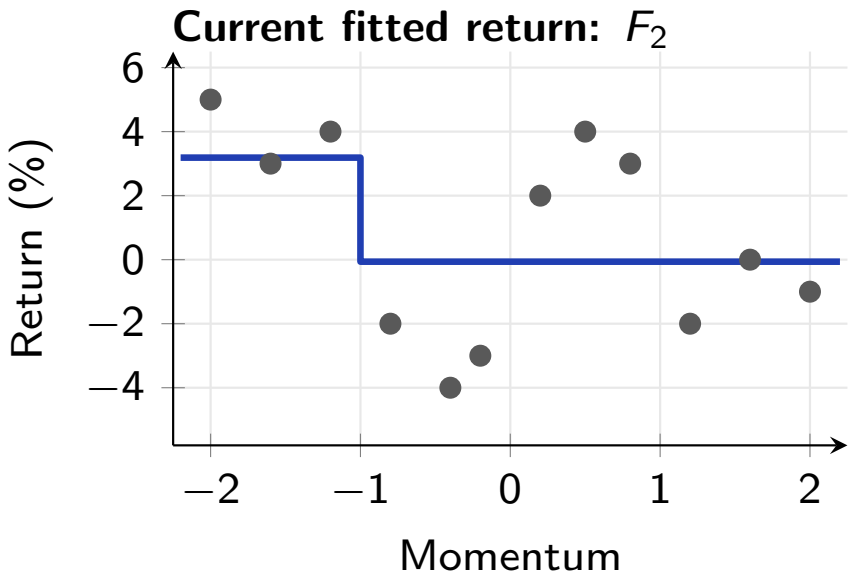


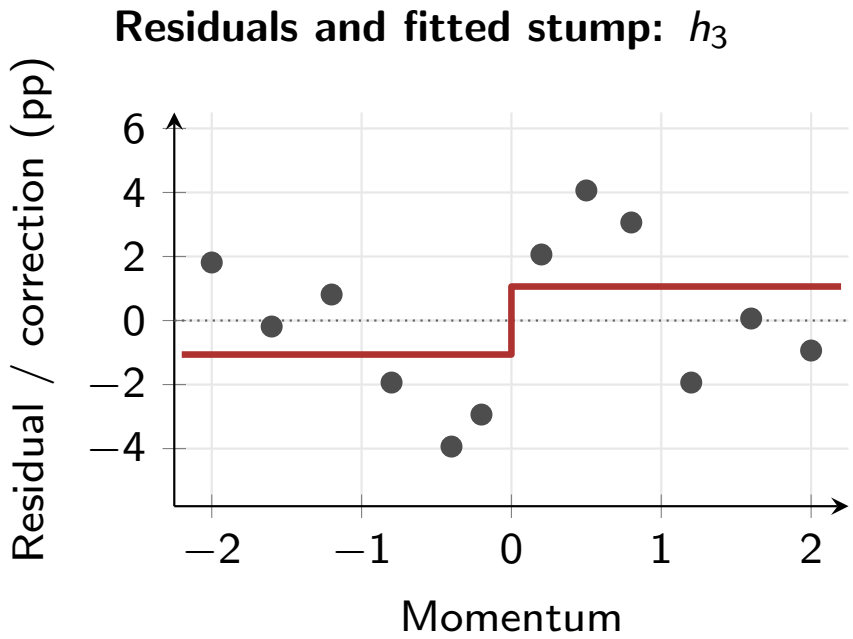


The prediction moves further toward the observations, but stocks to the right of $X = -1$ still receive the same fitted return.

Step 3: The Next Tree Finds a New Pattern

Fit the next stump to $e_i^{(2)} = r_i - F_2(X_i)$.





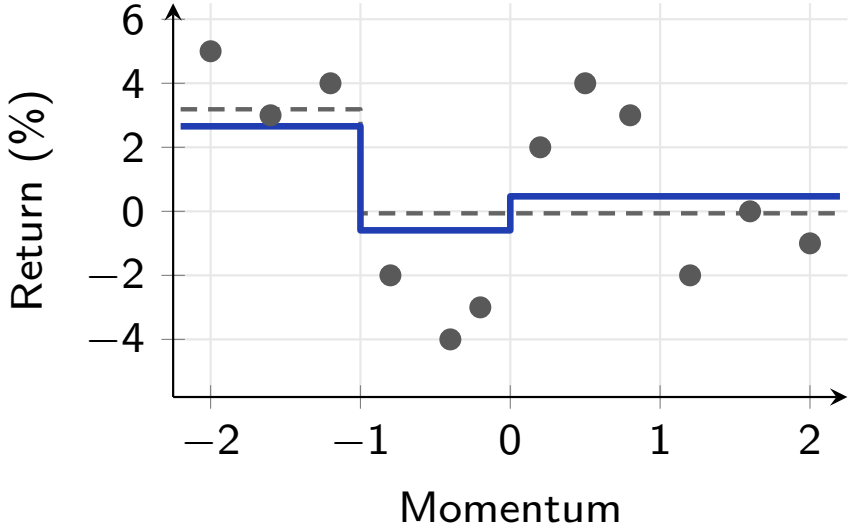
Q: What is different about this tree?

A: It splits at $X = 0$, lowering predicted returns to the left and raising them to the right.

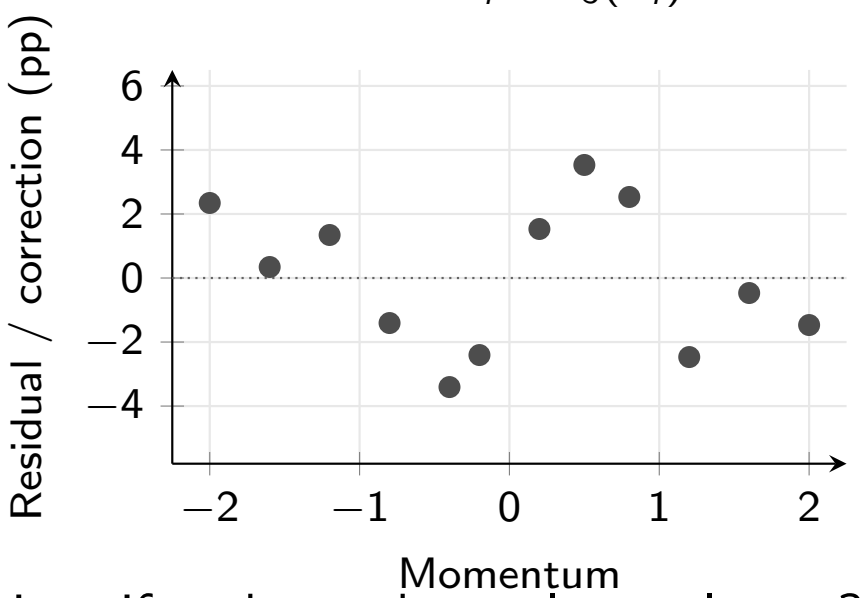
Step 3: Combining Stumps Creates More Regions

$$F_3(X) = \underbrace{F_0(X)}_{\text{constant}} + 0.5h_1(X) + 0.5h_2(X) + 0.5h_3(X).$$

Gray dashed: F_2; blue: F_3



New residuals: $r_i - F_3(X_i)$

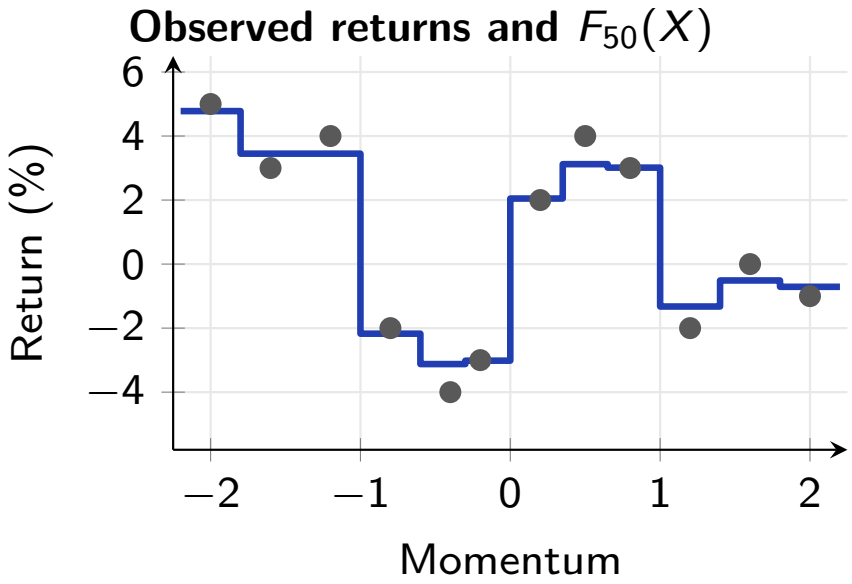


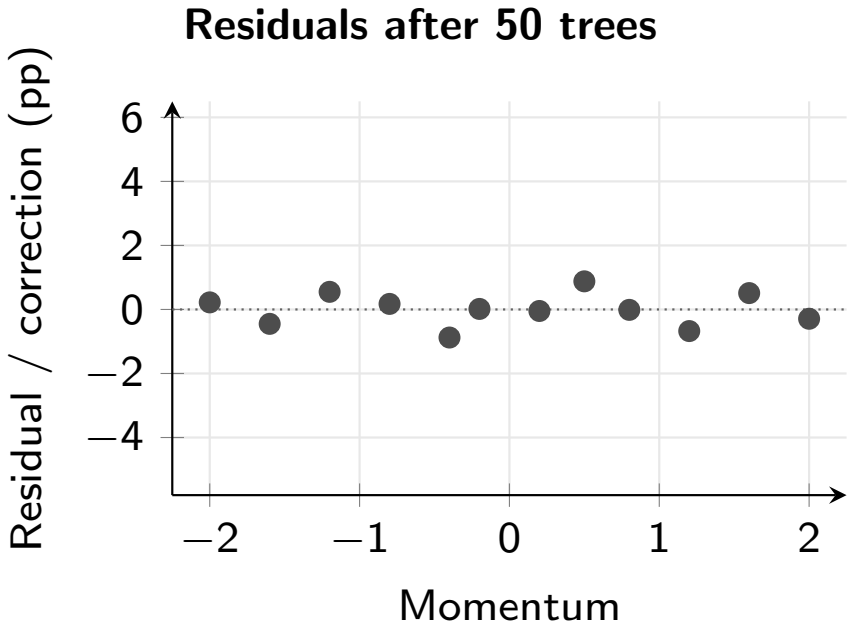
Q: How can the model have three fitted regions if each tree has only two leaves?

A: The *sum* of different step functions retains their split points.

After 50 Trees: A Flexible Step Function

Continue using the same $\nu = 0.5$ and one-split regression trees.





Q: Is this a smooth nonlinear curve?

A: No. The sum of step functions is still a step function, but many splits can approximate a nonlinear relationship closely.

This is an *in-sample* illustration, not evidence of out-of-sample predictive accuracy.

Why Is Each Tree's Prediction the Mean Residual?

Recall the regression tree criterion. In any candidate leaf R, the new tree predicts a constant c.

$$\min_c \sum_{i:X_i \in R} (e_i^{(m-1)} - c)^2.$$

This is minimized by

$$\hat{c}_R = \frac{1}{N_R} \sum_{i:X_i \in R} e_i^{(m-1)}.$$

CART selects the split that produces the largest reduction in the *sum of squared residuals* across the two leaves.

Q: How is this different from an ordinary regression tree?

A: The mechanics are the same, but the outcome being fitted is now the *current residual*, rather than the original return.

The Gradient-Boosting Algorithm: Squared Error

1. **Initialize** the prediction at the sample mean:

$$F_0(X) = \bar{r}.$$

2. For $m = 1, \ldots, M$:
   1. Compute current errors: $e_i^{(m-1)} = r_i - F_{m-1}(X_i)$.
   2. Fit a small regression tree $h_m(X)$ to those errors.
   3. Update:

$$\boxed{F_m(X) = F_{m-1}(X) + \nu h_m(X).}$$

Thus

$$F_M(X) = \bar{r} + \nu \sum_{m=1}^{M} h_m(X).$$

The trees are *added sequentially*; they are not averaged.

Why Is It Called *Gradient* Boosting?

We want predictions that minimize squared-error loss:

$$L(F) = \frac{1}{2}\sum_{i=1}^{n}[r_i - F(X_i)]^2.$$

For one observation, differentiate with respect to its fitted value:

$$\frac{\partial\left\{\frac{1}{2}[r_i - F(X_i)]^2\right\}}{\partial F(X_i)} = F(X_i) - r_i.$$

The *negative gradient* is

$$-\frac{\partial \ell_i}{\partial F(X_i)} = r_i - F(X_i) = e_i.$$

Q: Why does fitting a tree to residuals make sense?
A: For squared-error loss, residuals point in the direction that reduces the loss. The tree approximates those desired corrections.

Beyond Squared-Error Loss

For a more general loss $\ell(r_i, F(X_i))$, define the *pseudo-residual*:

$$z_i^{(m)} = - \left. \frac{\partial \ell(r_i, F)}{\partial F} \right|_{F=F_{m-1}(X_i)}.$$

- **Squared error:** pseudo-residuals equal ordinary residuals (up to a constant scaling convention).
- **Other losses:** trees follow different negative gradients, which need not equal $r_i - \hat{r}_i$.
- Practical boosting algorithms may also optimize the leaf updates for the chosen loss.

Key idea: gradient boosting adds new trees in directions that seek to reduce a specified loss function.

Learning Rate and Number of Trees

The learning rate ν determines how much of each tree's correction we add:

$$F_m(X) = F_{m-1}(X) + \underbrace{\nu}_{\text{shrinkage}} h_m(X), \qquad 0 < \nu \leq 1.$$

- **Large ν:** each tree makes a relatively large change.
- **Small ν:** each step is more cautious, and typically more trees are needed to achieve the same training fit.
- **Number of trees M:** determines how many corrections are accumulated.

Tree Depth: Main Effects and Interactions

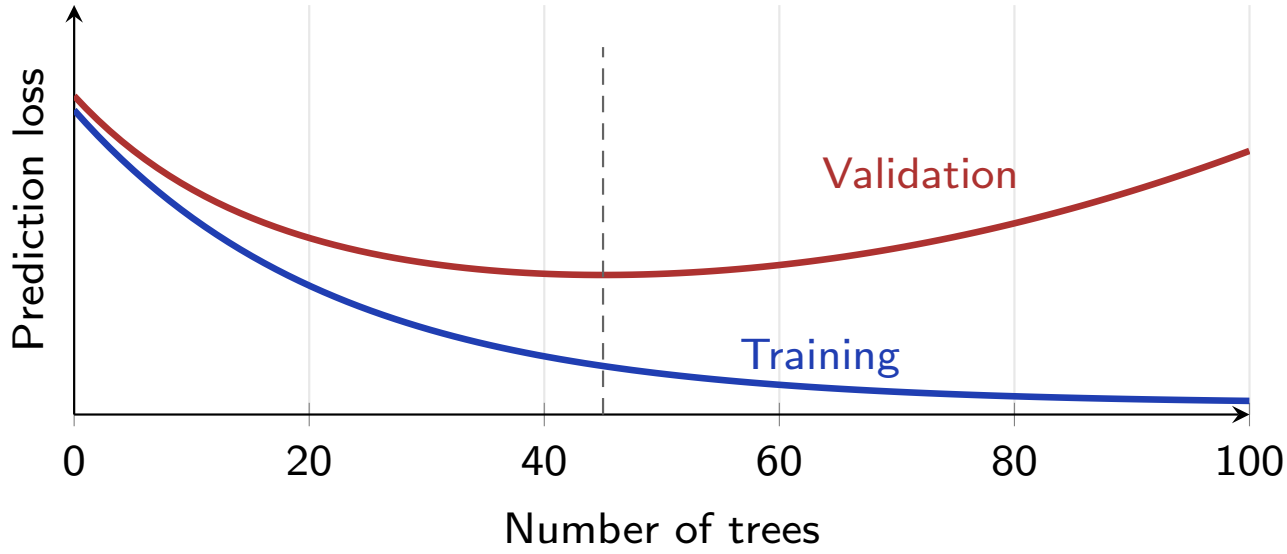
Our illustrations used **stumps**: each tree has one split.
With multiple features, one stump depends on only one feature. An ensemble of stumps therefore has an *additive* form:

$$F_M(X_1, \ldots, X_K) = c + \sum_{k=1}^{K} g_k(X_k).$$

Deeper trees allow successive splits on different features within the same tree, capturing interactions.
Example: the momentum–return relationship might depend on whether volatility is high or low.
Q: What is the tradeoff as tree depth increases?
A: Greater flexibility, including interactions, but also more opportunities to fit noise.

Overfitting and Early Stopping

More boosting iterations generally reduce *training* squared error, but need not improve predictions on new observations.



Q: When should we stop adding trees?
A: Near the minimum *validation* loss (early stopping), not where training loss is smallest.

Other Hyperparameters

- **Depth / number of leaves:** how complex an individual correction can be; deeper trees can capture interactions.
- **Minimum observations per leaf:** prevents corrections based on extremely small groups of stocks.
- **Subsampling:** fit each new tree using a random subset of training observations (*stochastic* gradient boosting); this can reduce overfitting.
- **Learning rate and number of trees:** typically chosen jointly.

Implementation principle: tune using validation data or cross-validation appropriate to the investment application, and reserve an untouched test set for final evaluation.

Random Forests vs. Gradient Boosting

Both methods combine regression trees, but construct their ensembles differently.

	Random forest	**Gradient boosting**
How trees are fit	Many trees trained separately, with randomness	Trees trained sequentially
What each tree fits	Original return	Current residual (for squared error)
Combining trees	Average predictions	Add scaled corrections
Main intuition	Reduce variability through averaging	Improve the fit stage by stage

Q: Can boosting overfit even if each tree is small?
A: Yes. Many successive corrections can eventually fit noise.

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
5. Gradient Boosting
6. Model Comparison

Comparing Methods

Method	Key Idea	Flexibility	Interpretability
GAM	Fixed nonlinear basis (splines)	Medium	High
Random Forest	Average many trees	High	Medium
Gradient Boosting	Sequential error correction	Very High	Medium
Neural Network	Learned nonlinear features	Very High	Low

Key takeaway:

- All models estimate the same object: $r_i = f(X_i)$
- They differ mainly in how they approximate $f(\cdot)$

Choosing a Method: Theory and EDA

Q: What does theory or EDA suggest about $f(X)$?

- **Smooth, largely additive relationships?**
  - Consider a GAM.
- **Thresholds, regimes, or feature interactions?**
  - Consider Random Forests or Gradient Boosting.
- **Complex combinations of many features?**
  - Consider Neural Networks.

Other considerations:

- **Sample size:** Limited data or weak signals may favor simpler, more heavily regularized models.
- **Extrapolation:** Tree-based models generally cannot extrapolate trends beyond the training range. Theory-guided functional forms may be more appropriate.

Key takeaway: Theory and EDA help narrow our choices. Out-of-sample performance helps determine which method works best.

Outline

1. Motivation
2. Generalized Additive Models (GAMs)
3. Neural Networks
4. Random Forests
   - Regression Trees
   - Bagging
   - Random Forests
5. Gradient Boosting
6. Model Comparison